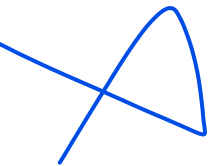


Tree

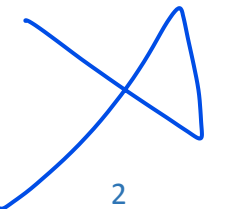


TREE PART II

Euler Tour Traversal

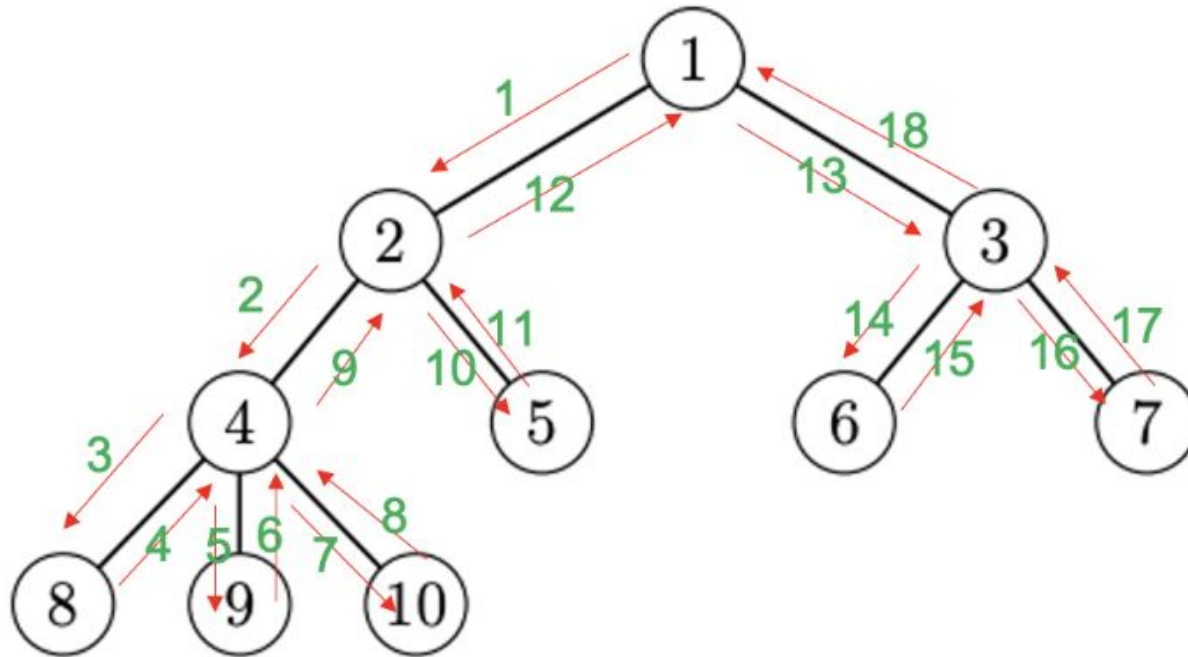
Property of Proper Binary Tree

Implementing Binary Trees



Euler Tour Traversal

- In a graph G , Euler tour is a path through the graph that visits **every edge** exactly once.

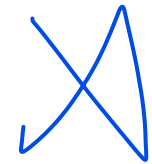


Euler tour:

1	2	4	8	4	9	4	10	4	2	5	2	1	3	6	3	7	3	1
---	---	---	---	---	---	---	----	---	---	---	---	---	---	---	---	---	---	---

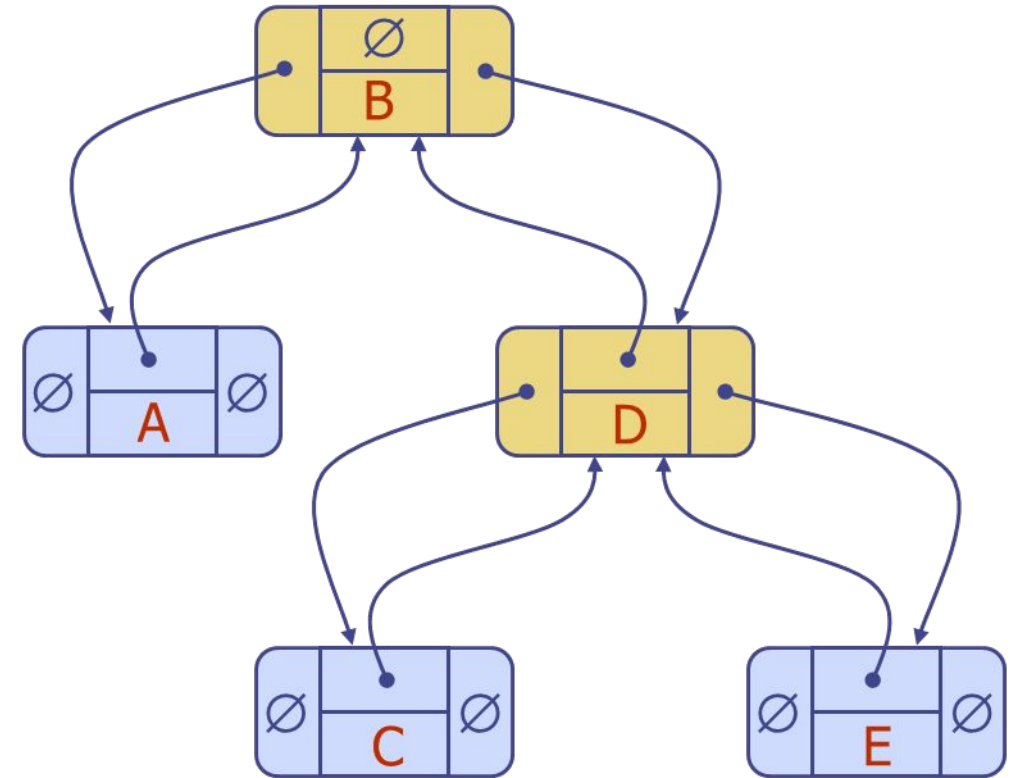
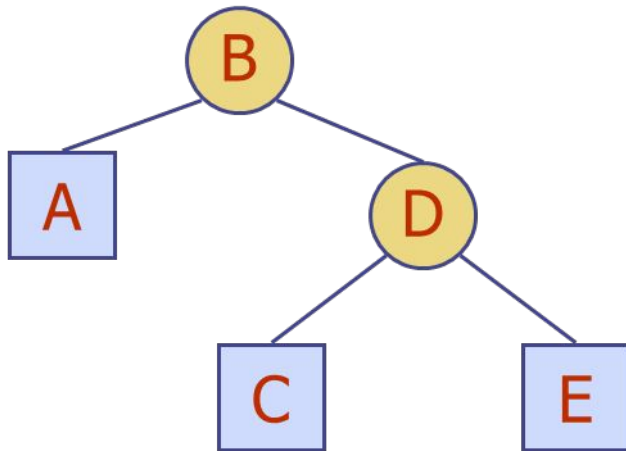
A Property of Proper Binary Trees

- A (proper) binary tree is a tree with the following properties:
 - Each internal node has two children
 - The children of a node are an ordered pair
- In a (proper) binary tree T , the number of external nodes is 1 more than the number of internal nodes.
- Proof:
 - Use Induction



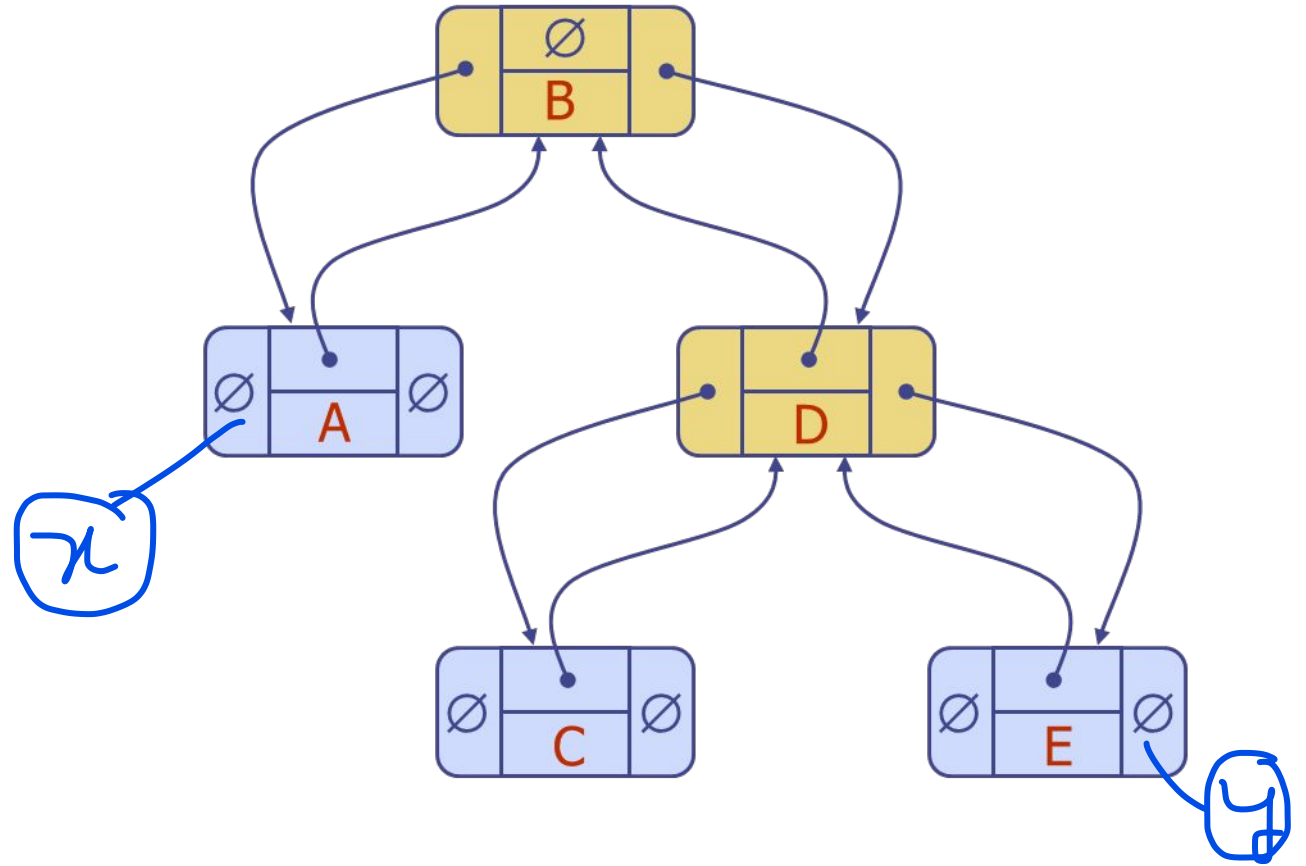
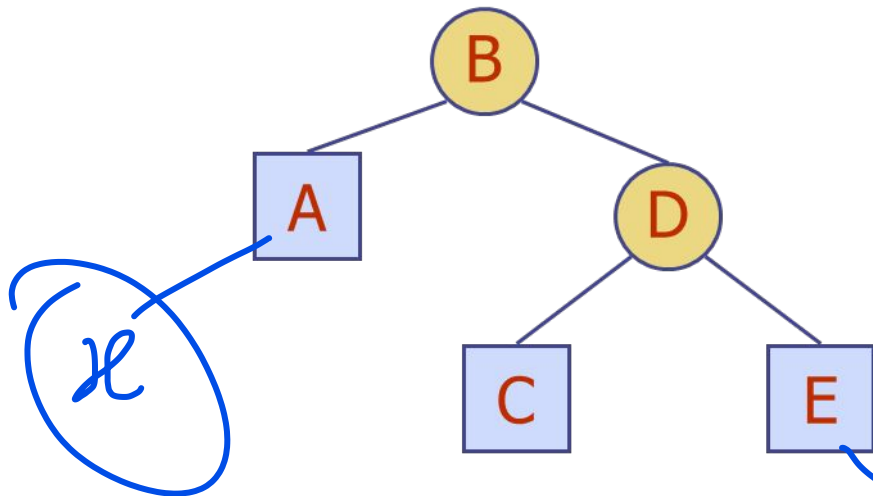
Implementing Binary Trees

```
struct TreeNode{  
    int value;  
    struct TreeNode* parent;  
    struct TreeNode* left;  
    struct TreeNode* right;  
};
```



Implementing Binary Trees

```
struct TreeNode{  
    int value;  
    struct TreeNode* parent;  
    struct TreeNode* left;  
    struct TreeNode* right;  
}
```

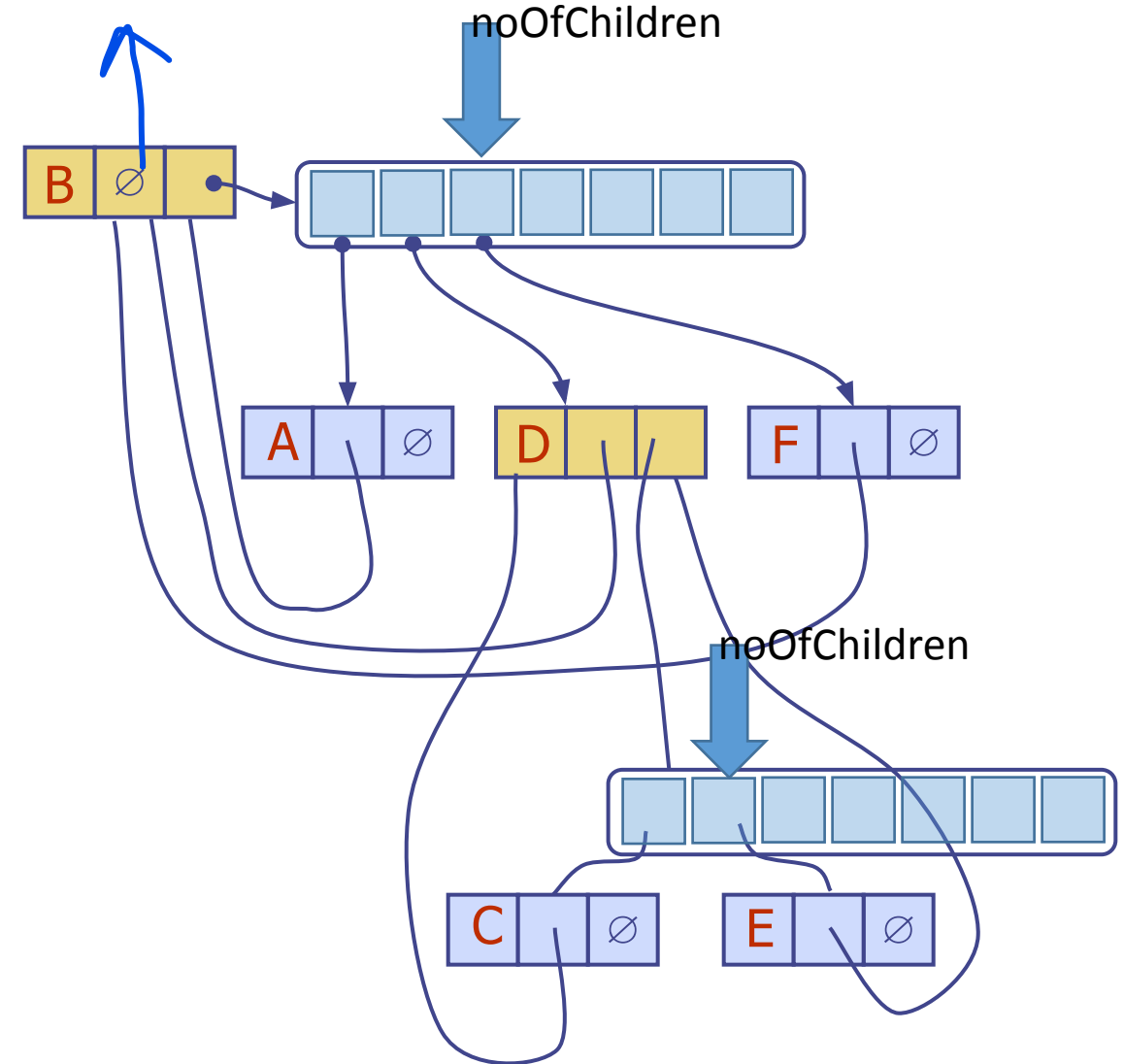
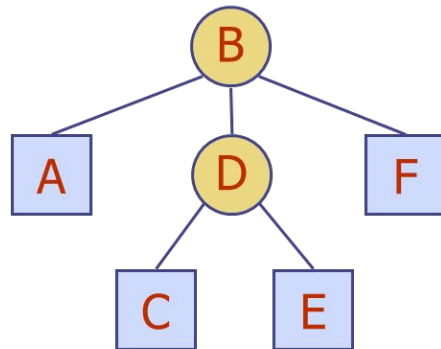


Write the necessary code to add a left child 'X' to A and a right child 'Y' to E

Implementing General Trees

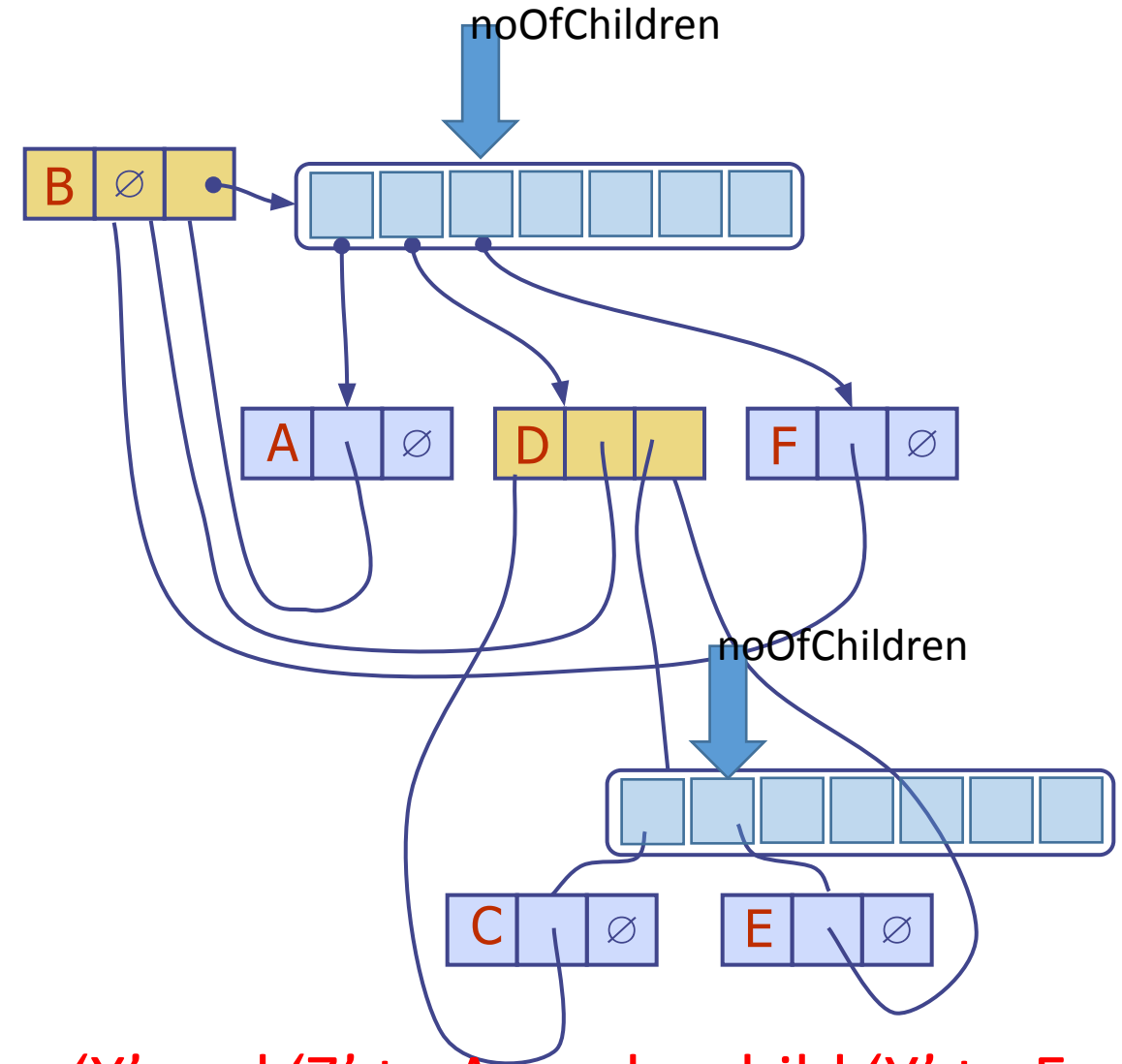
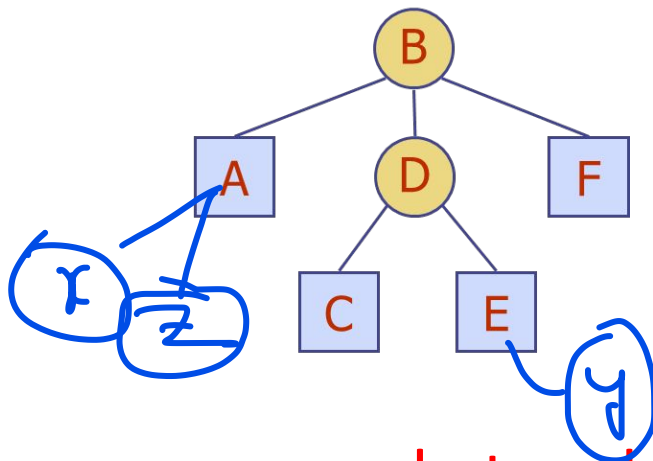
```
struct Container{  
    struct TreeNode* childrenArr[7];  
    int noOfChildren; 3  
}
```

```
struct TreeNode{  
    int value; B  
    struct TreeNode* parent;  
    struct Container* childrenContainer;  
}
```



Implementing General Trees

```
struct Container{  
    struct TreeNode* childrenArr[7];  
    int noOfChildren;  
}  
  
struct TreeNode{  
    int value;  
    struct TreeNode* parent;  
    struct Container* childrenContainer;  
}
```



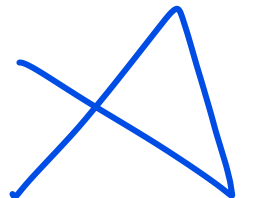
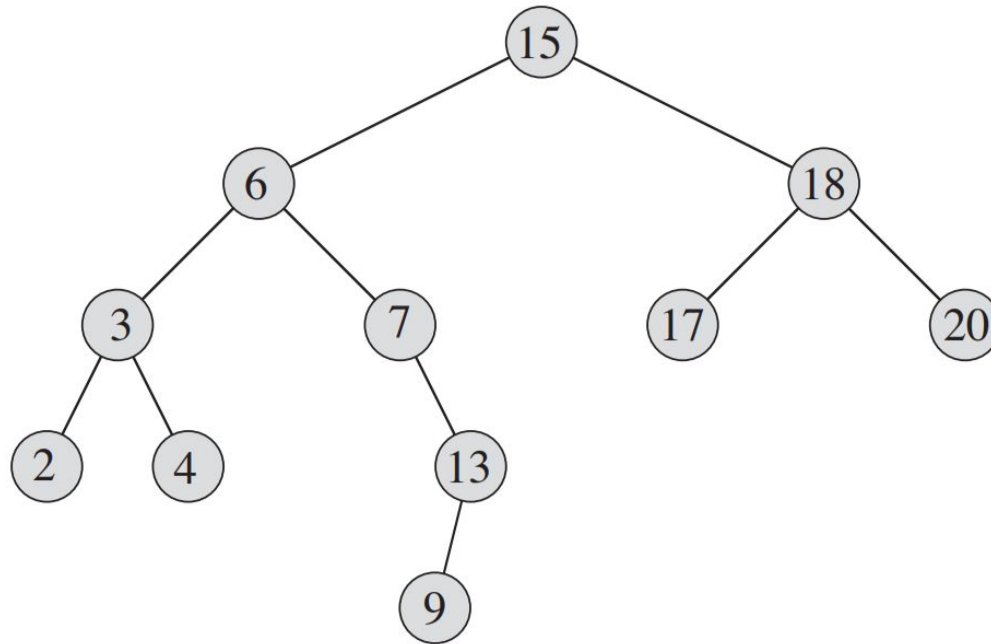
Write the necessary code to add children 'X' and 'Z' to A; and a child 'Y' to E

TREES PART - III

Binary Search Tree

Recall This From DM

- Previously we performed only insertion



Binary Search Tree - Intro

- A binary search tree (BST) is a node-based **binary** tree data structure which has the following properties:
- The left subtree of a node contains only nodes with **keys (values) less** than the node's key.
- The right subtree of a node contains only nodes with **keys greater than or equal** to the node's key.
- Both the **left** and **right subtrees** must also be **binary search trees**.

Two Things To Keep In Mind

1. Terms and complexity

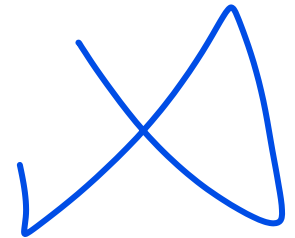
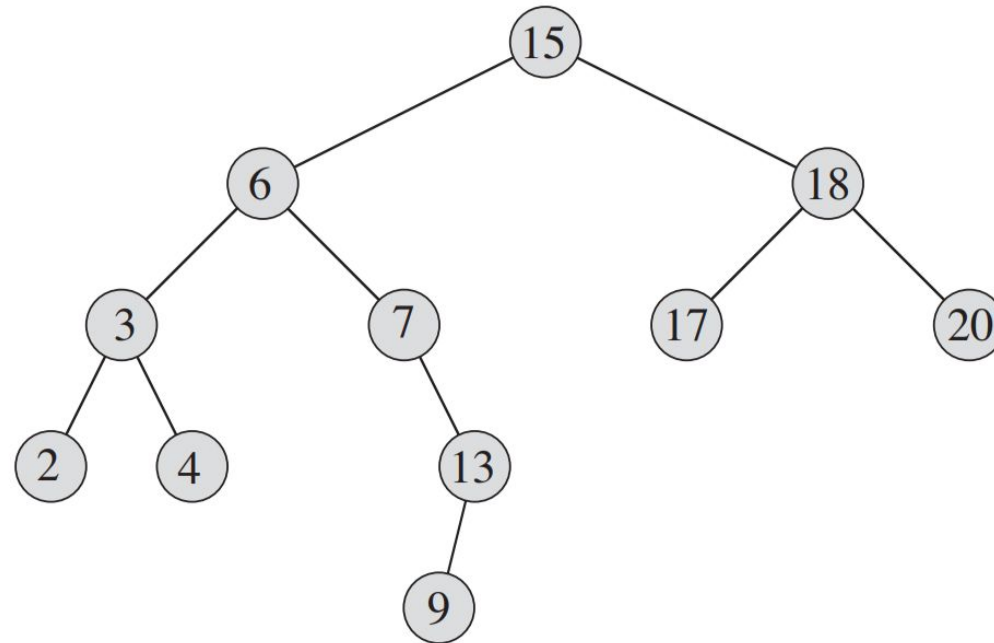
- It is a **binary** tree represented by a **linked** data structure
- It combines the advantage of an array - the ability to do a **binary search** with the advantage of a **linked list** its dynamic size
- The efficiency of all of the operations is $O(h) = O(\log n)$, only if the tree is reasonably height-balanced
- Each node contains at least the following **fields**
 - key: an identifying field
 - left: pointer to a left child (may be NIL)
 - right: pointer to a right child (may be NIL)
 - p: pointer to the parent node (NIL for root)

Two Things To Keep In Mind

2. Binary Search Property

For any node x

- $\text{key}[y] < \text{key}[x]$ if y in left subtree of x
- $\text{key}[y] \geq \text{key}[x]$ if y in right subtree of x

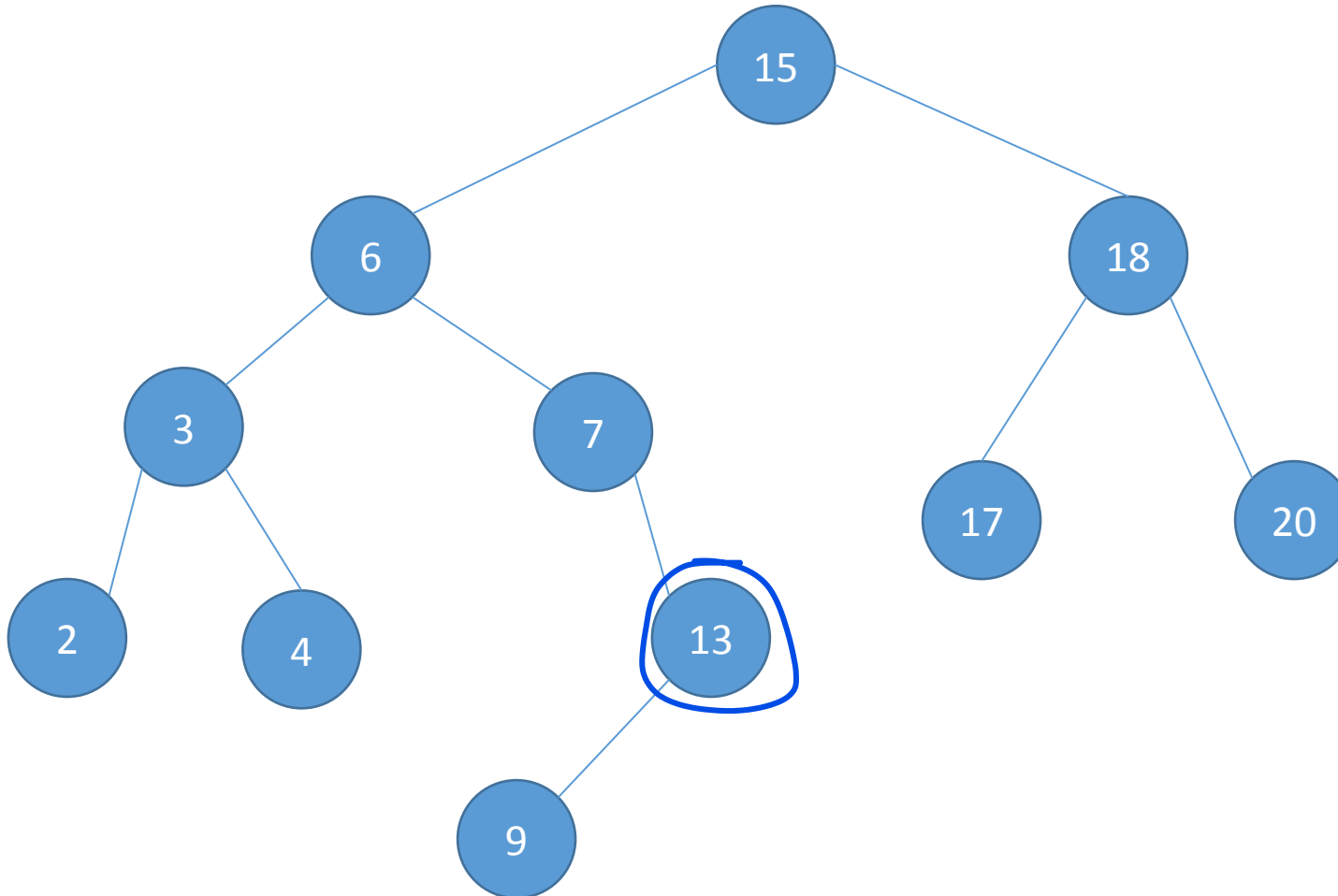


Binary Search Tree Operations

- The search tree data structure supports many dynamic-set operations, including
 - SEARCH
 - MINIMUM
 - MAXIMUM
 - PREDECESSOR
 - SUCCESSOR
 - INSERT
 - DELETE.

Search

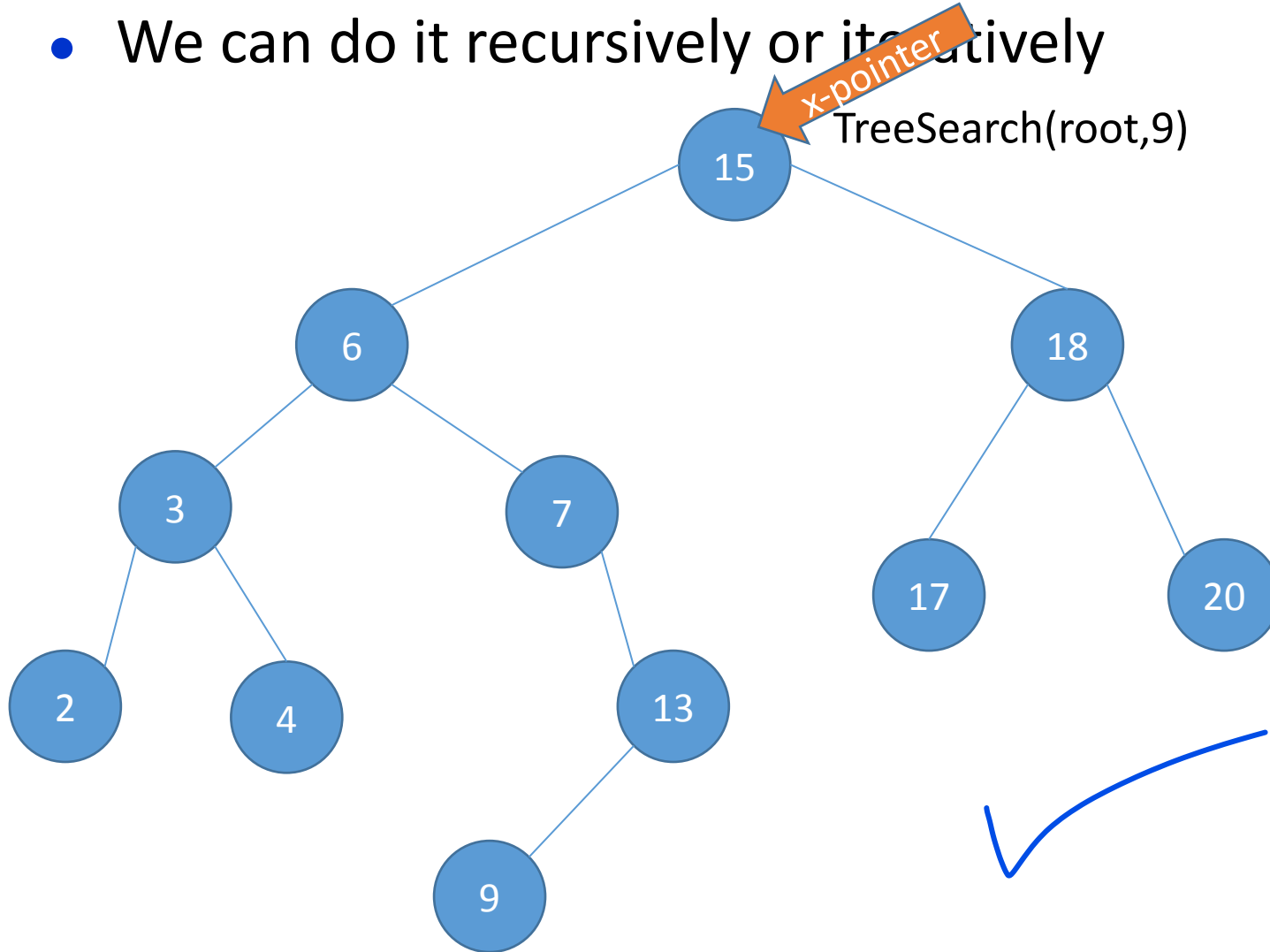
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search

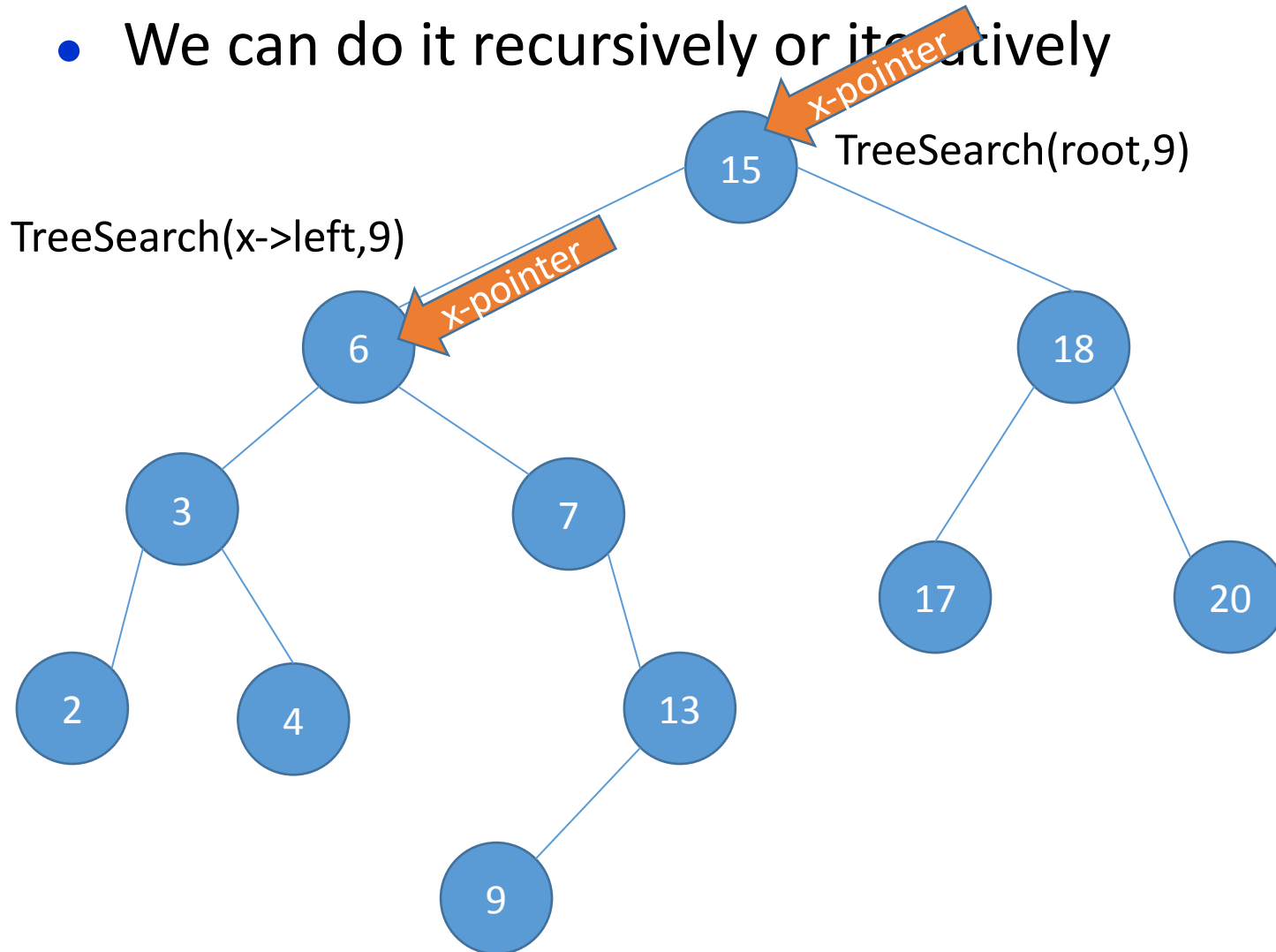
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search

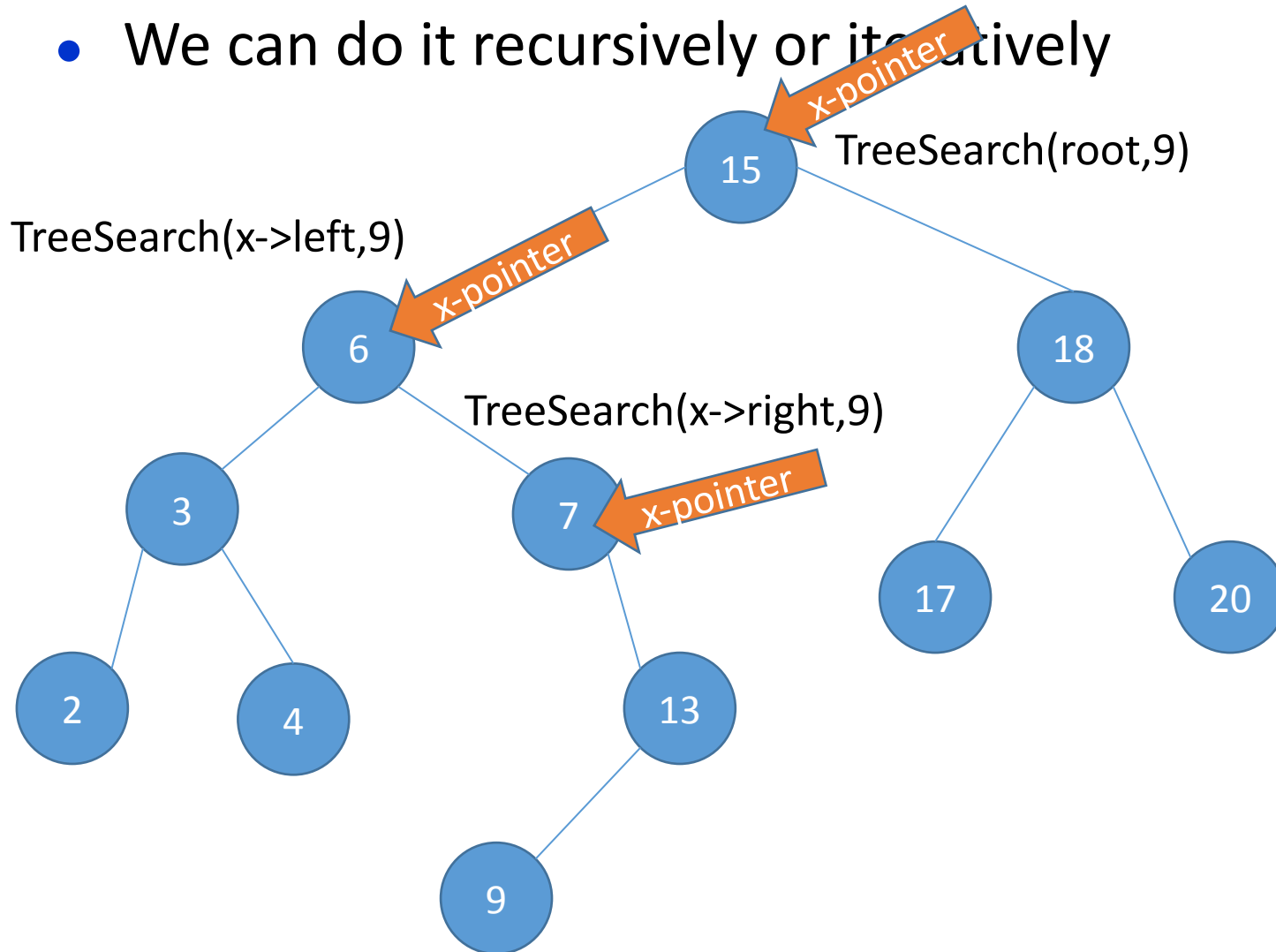
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search

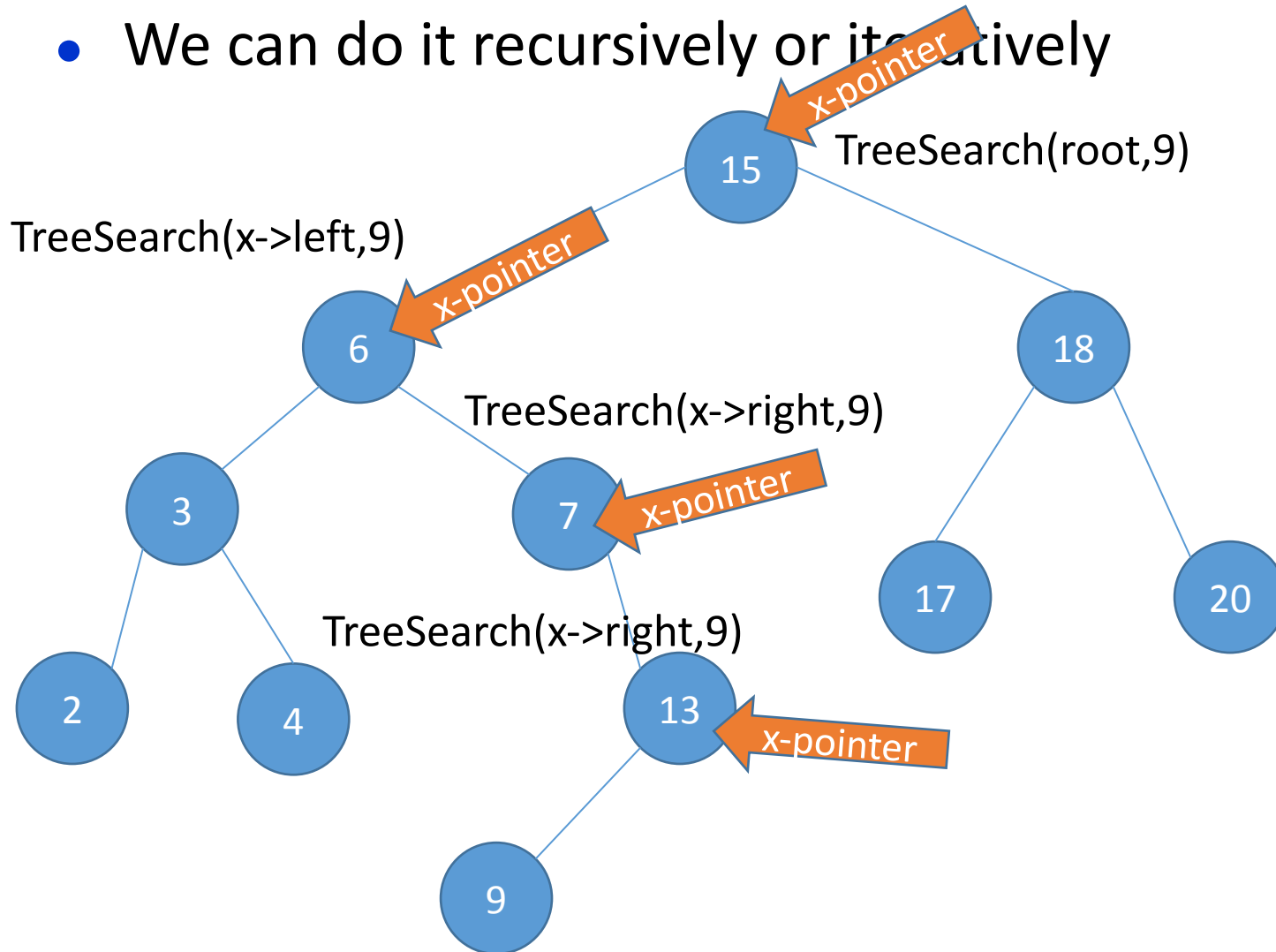
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search

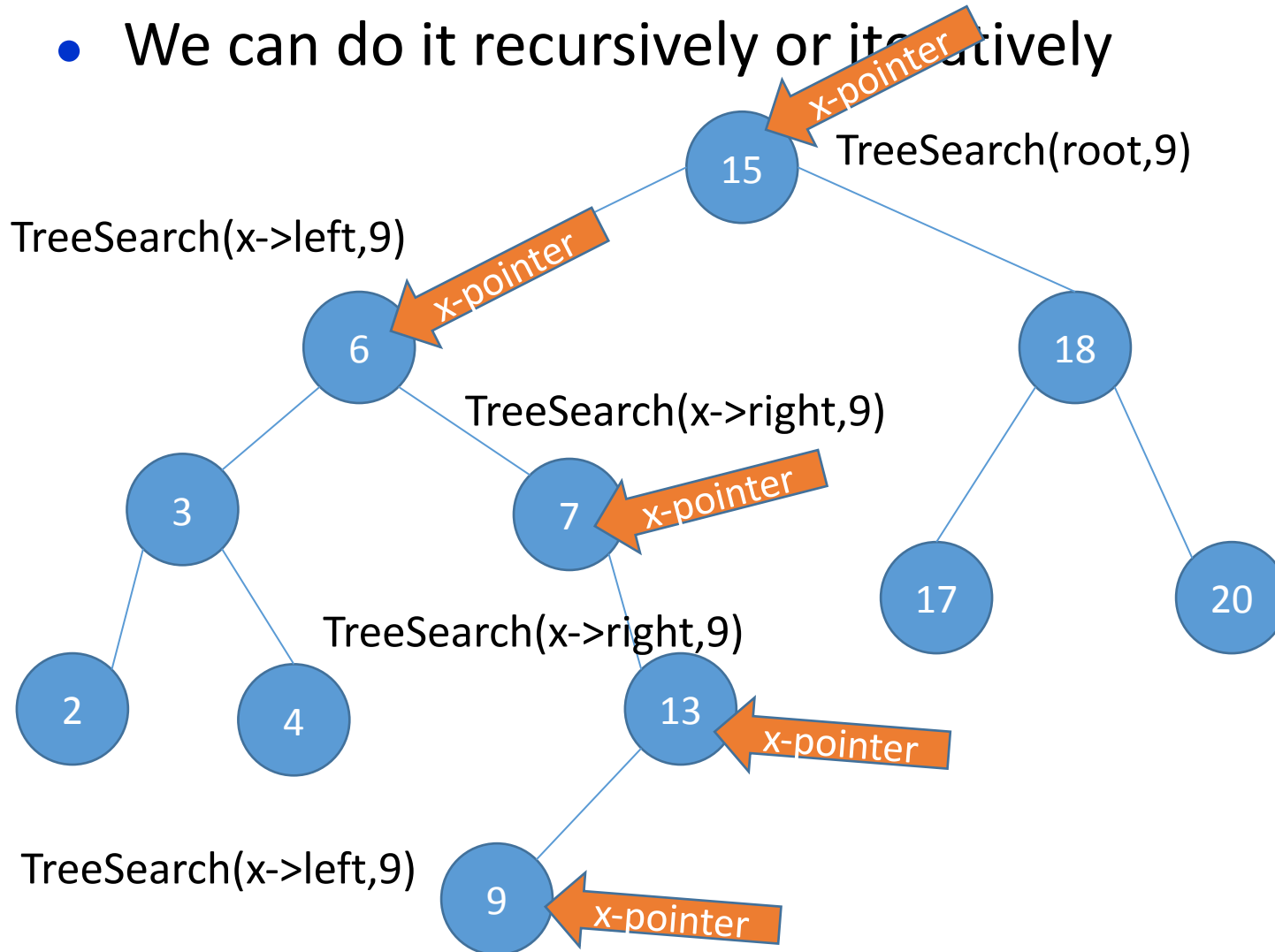
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search

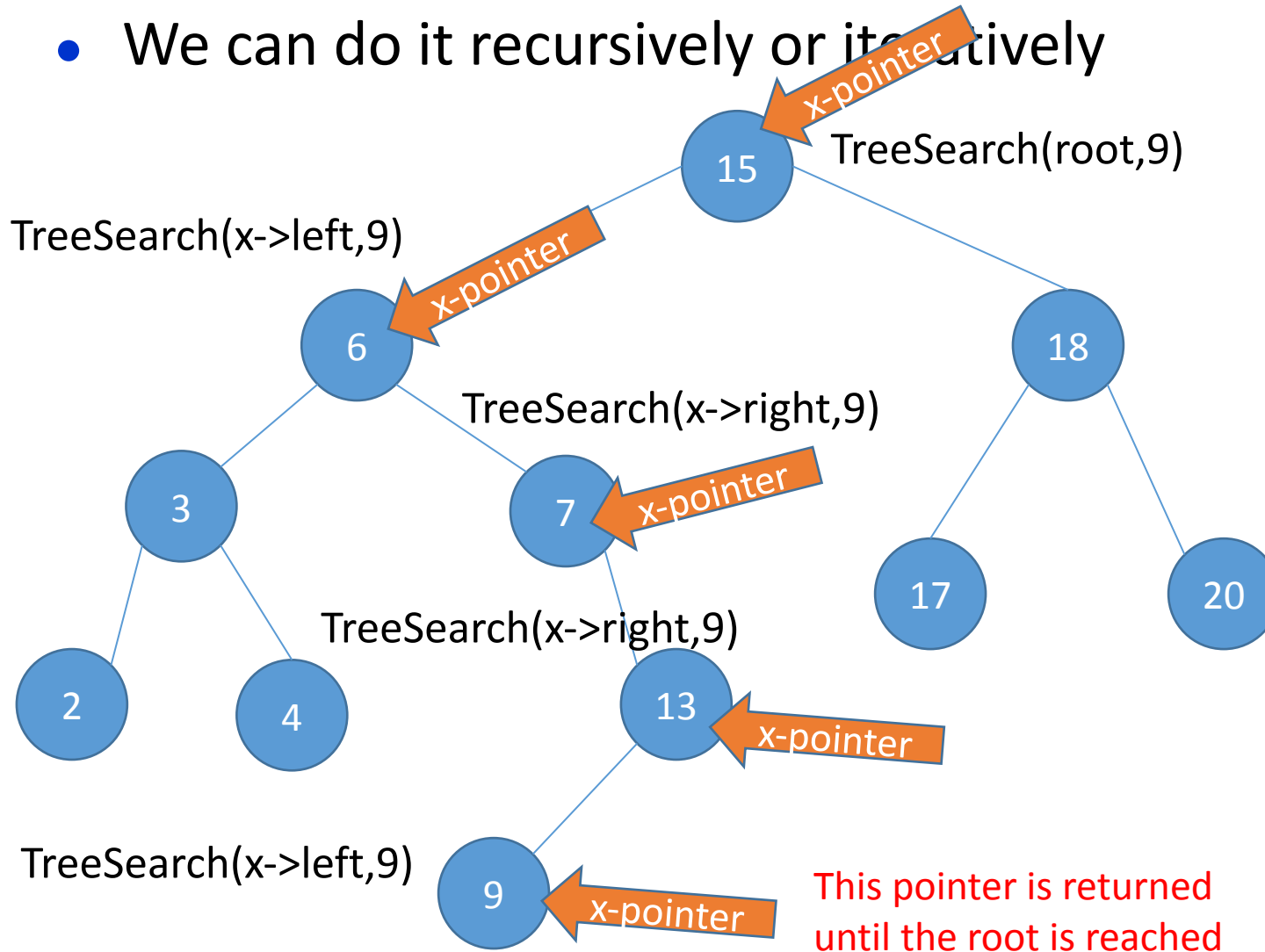
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search

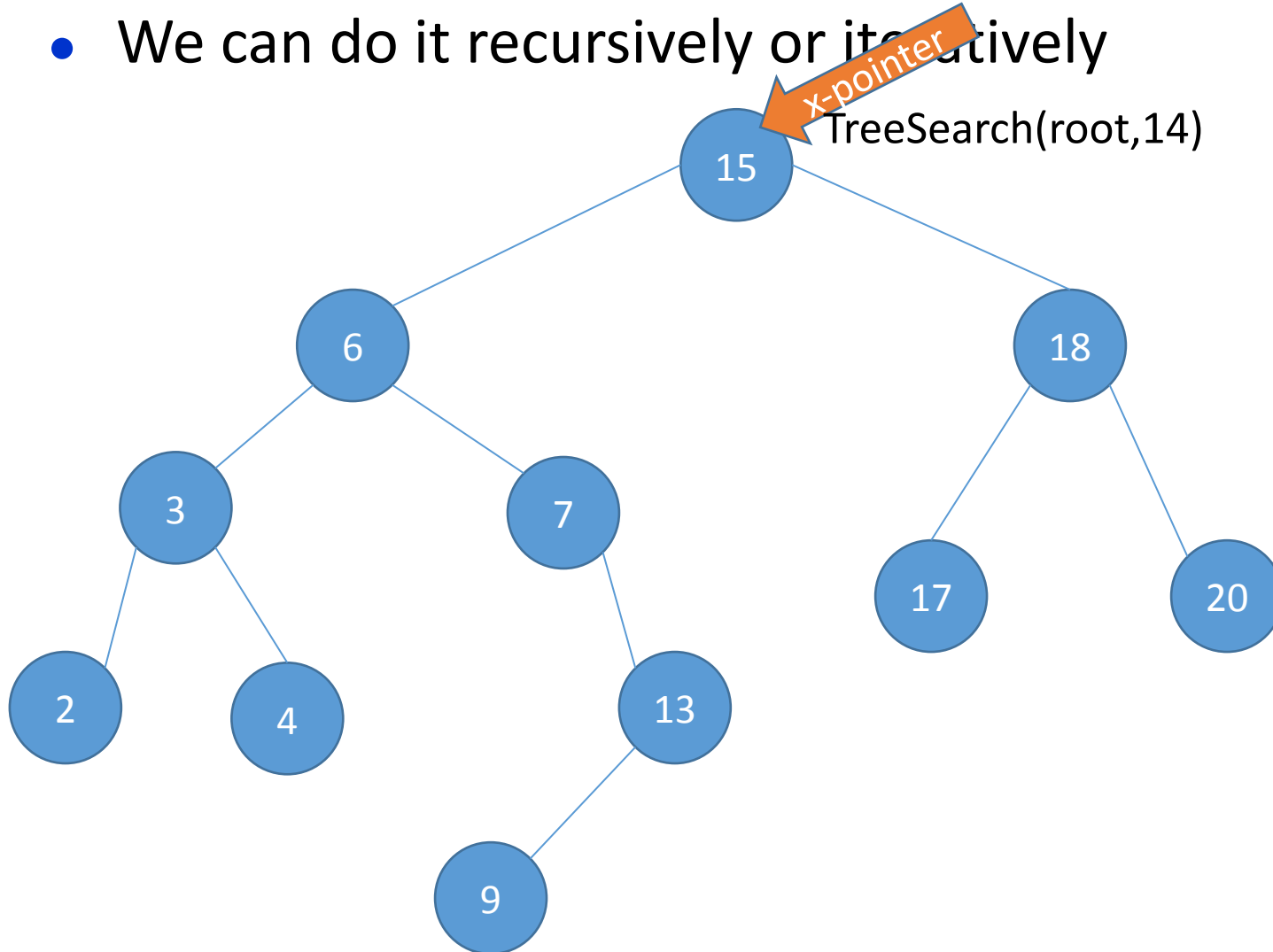
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search

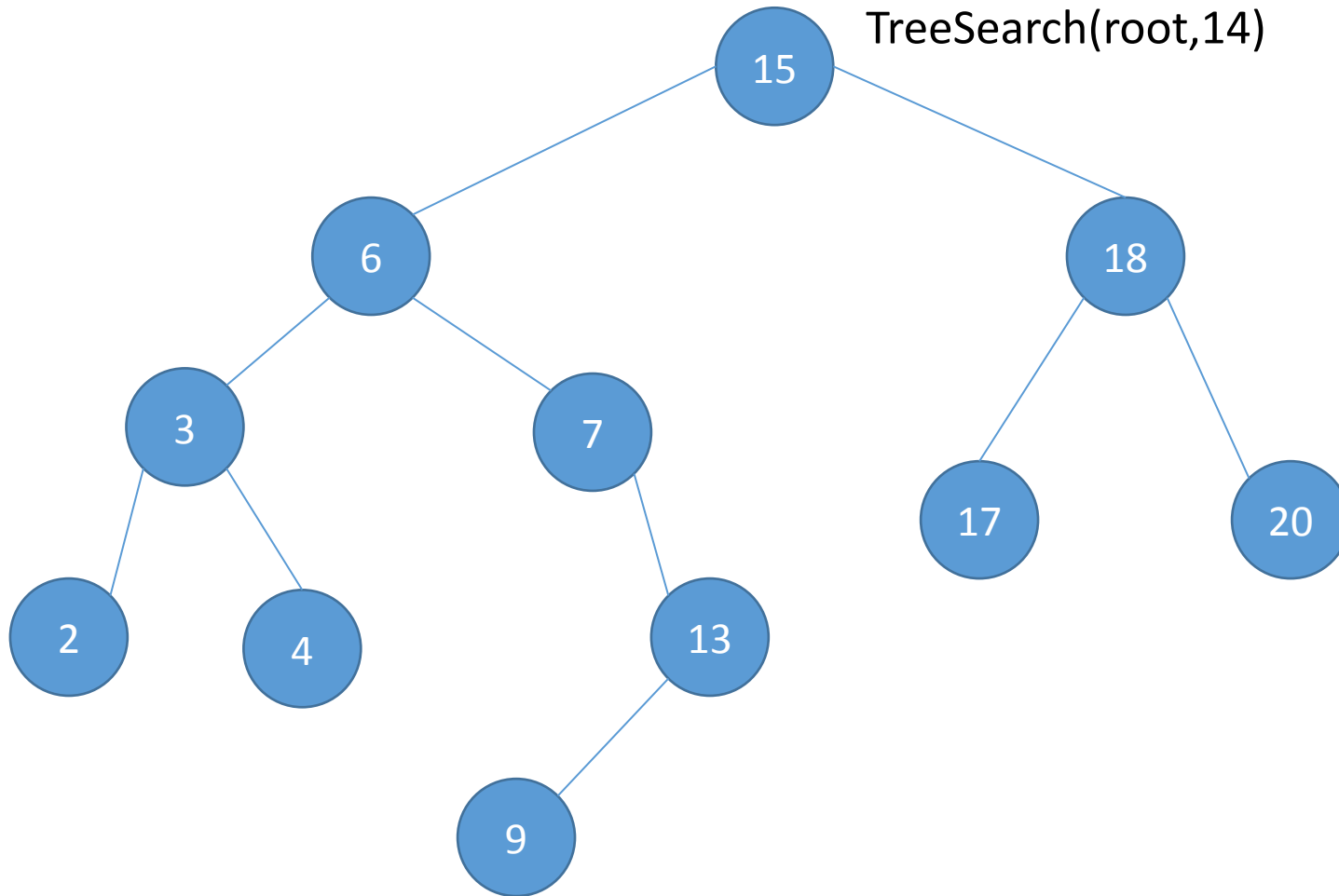
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

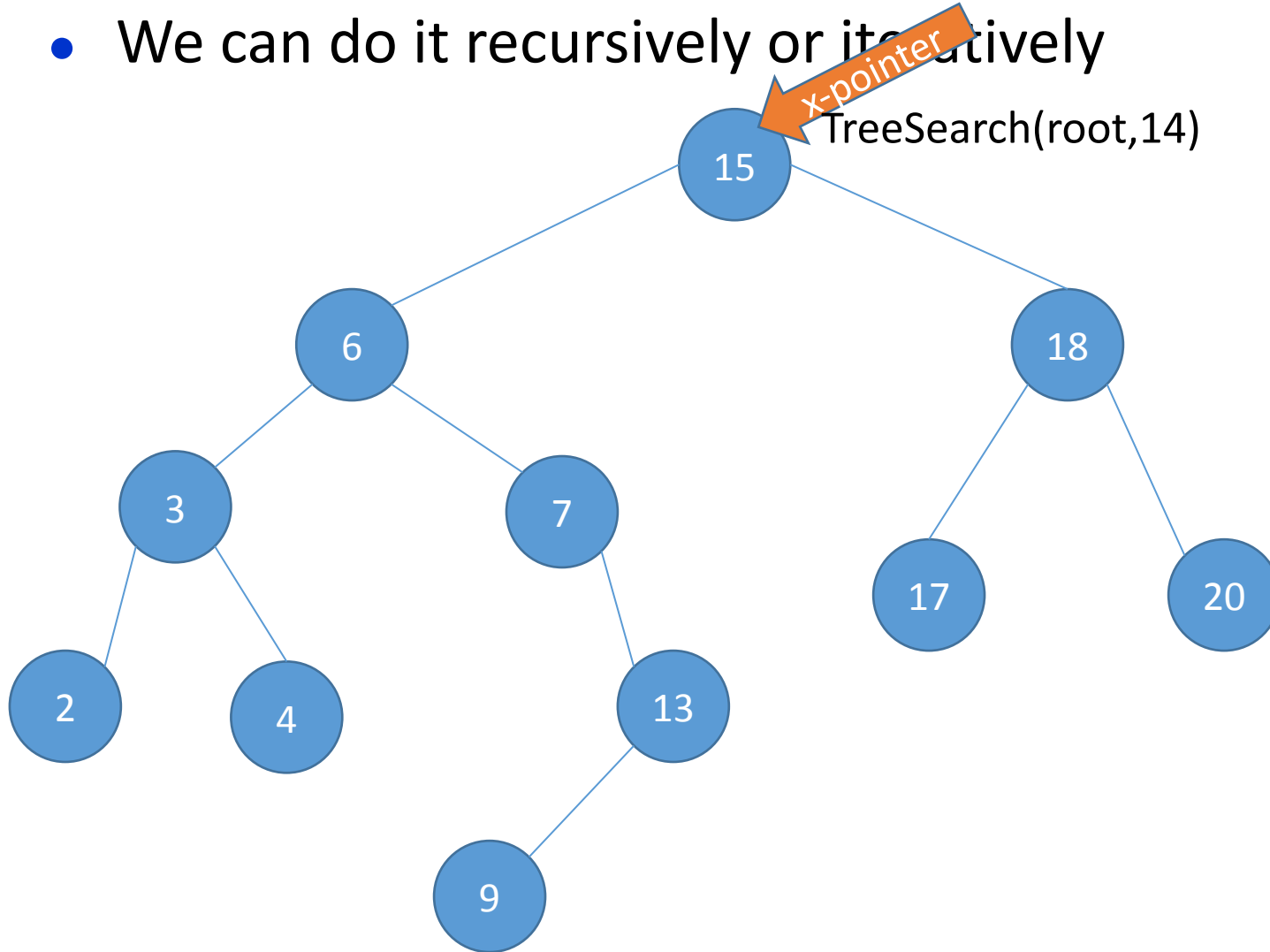
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

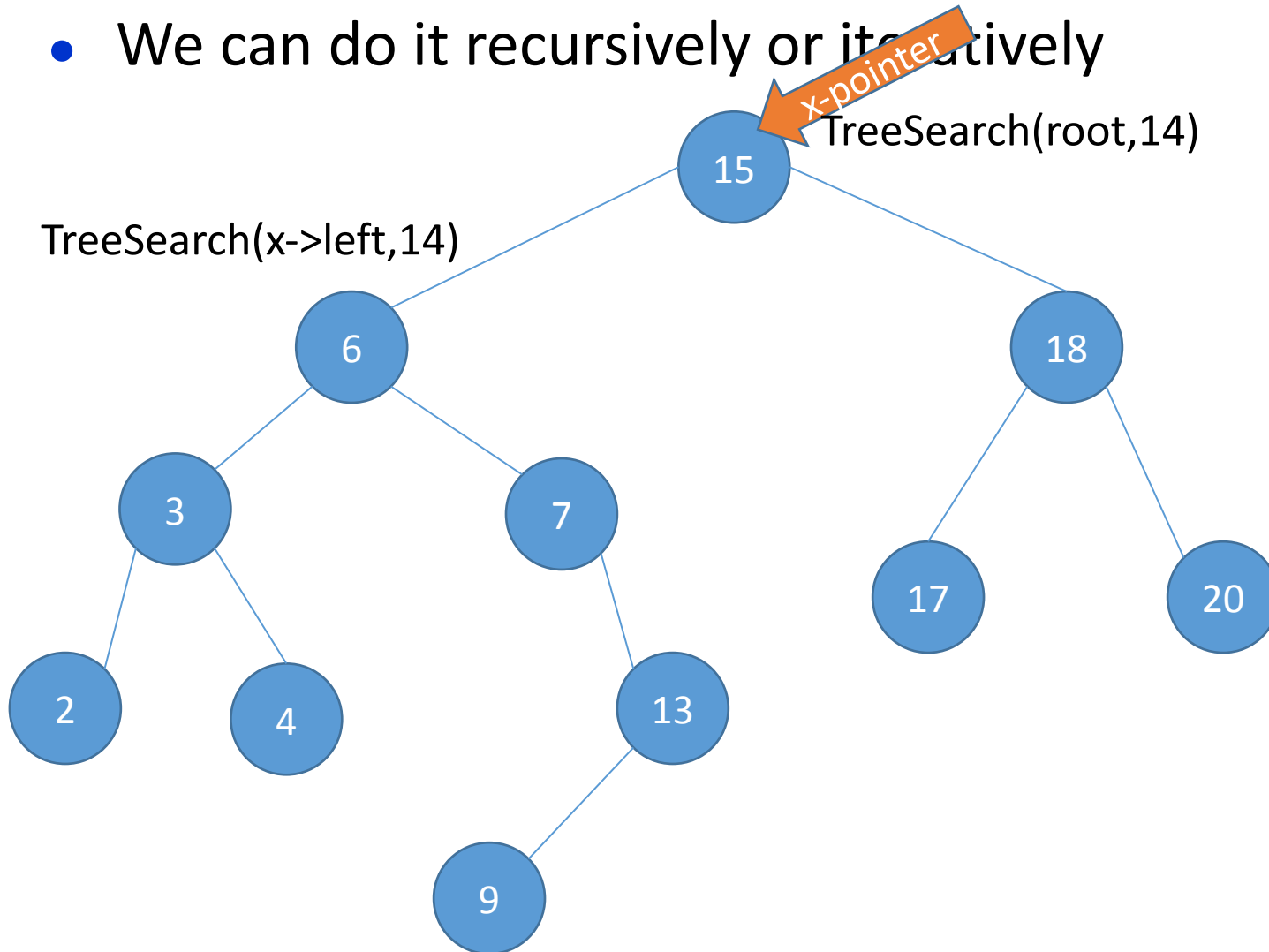
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

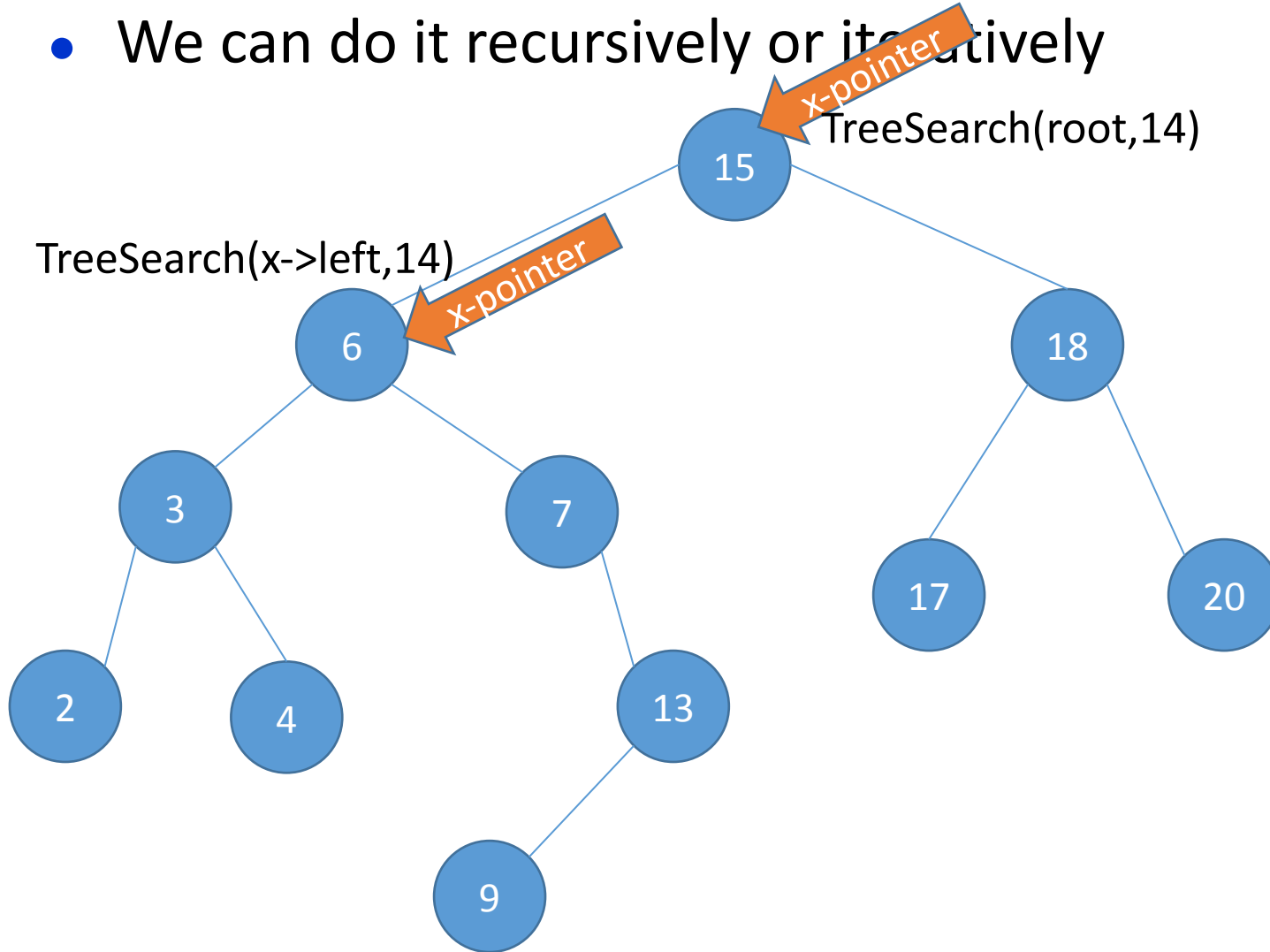
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

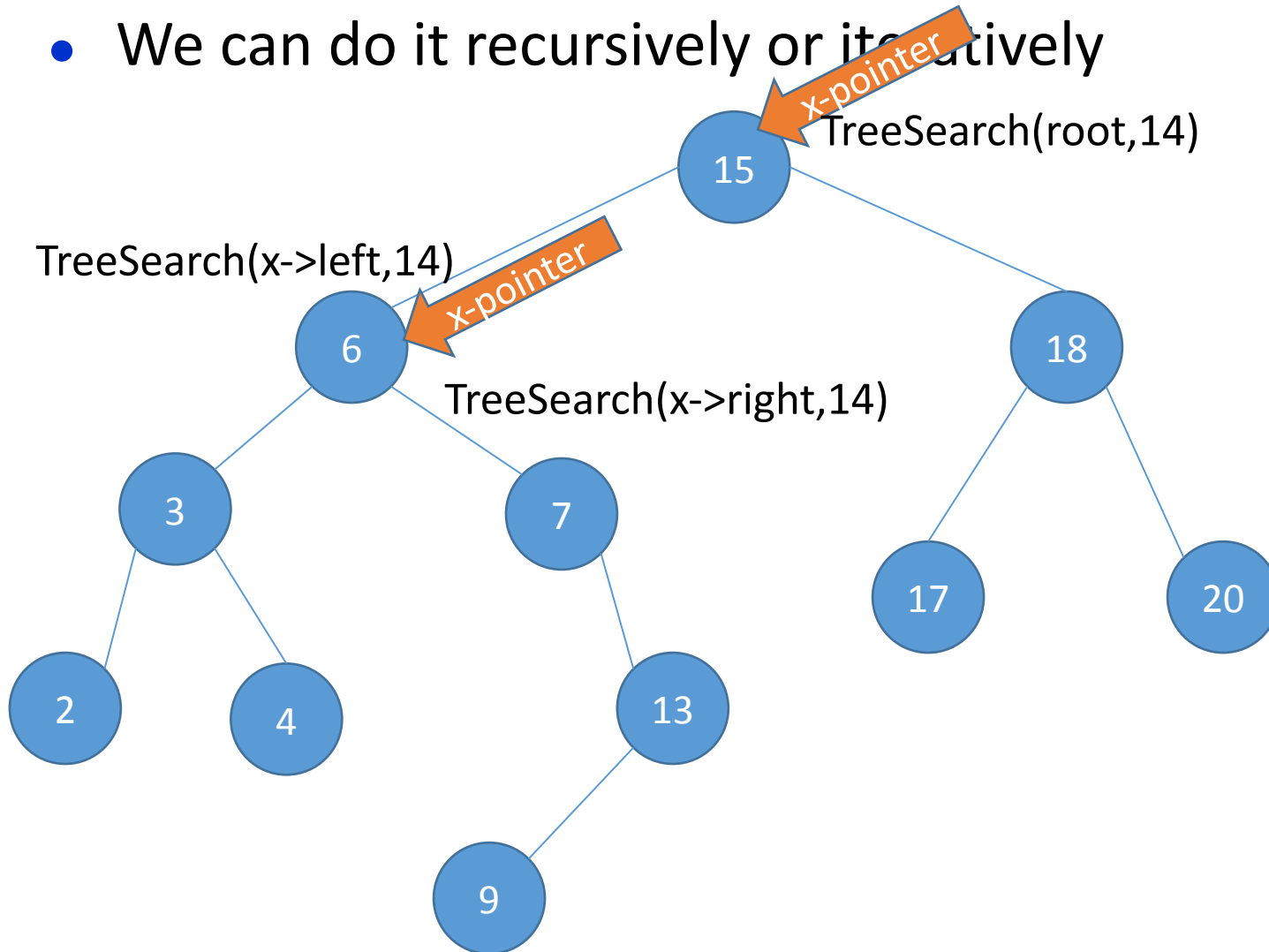
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

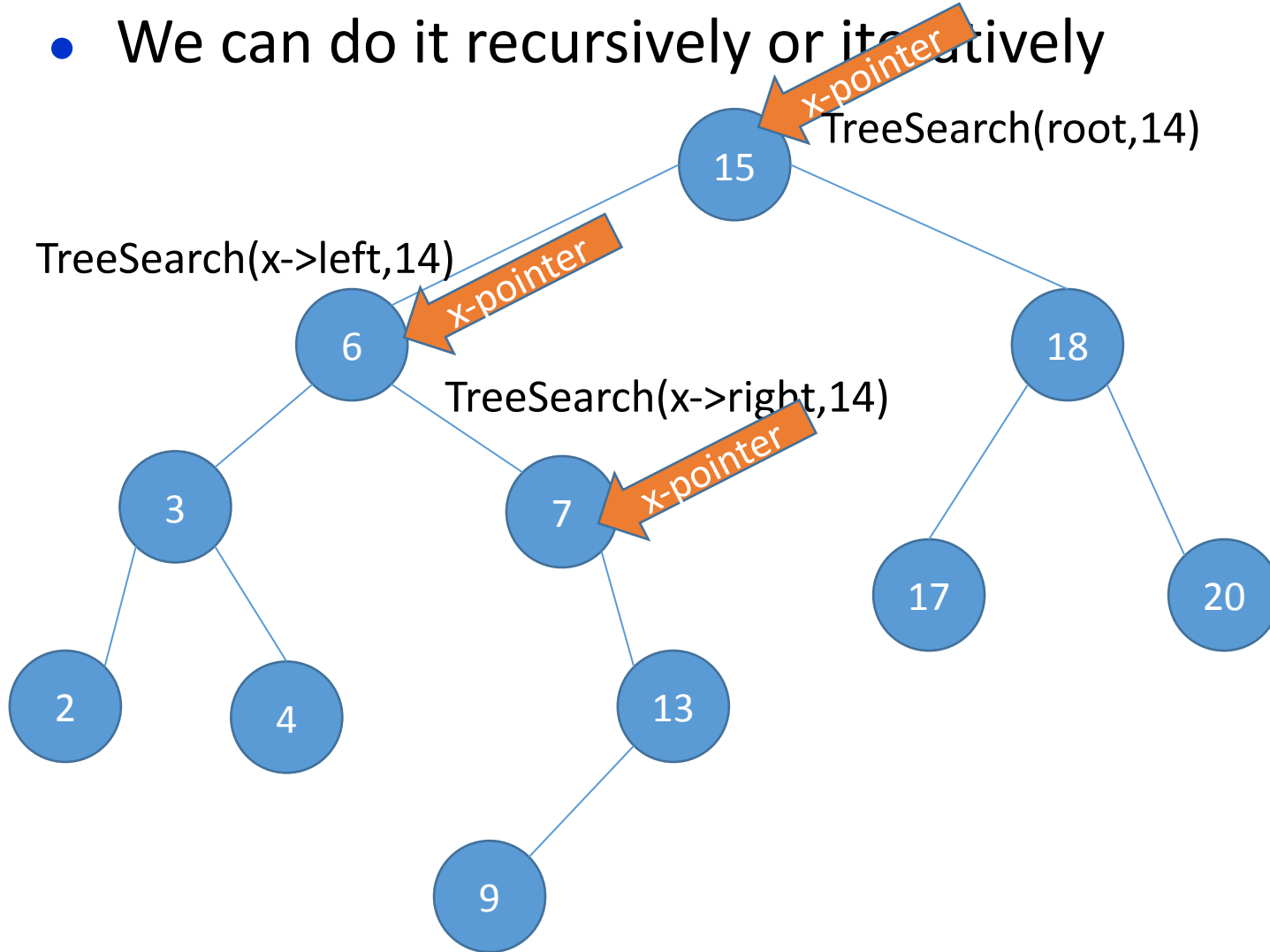
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

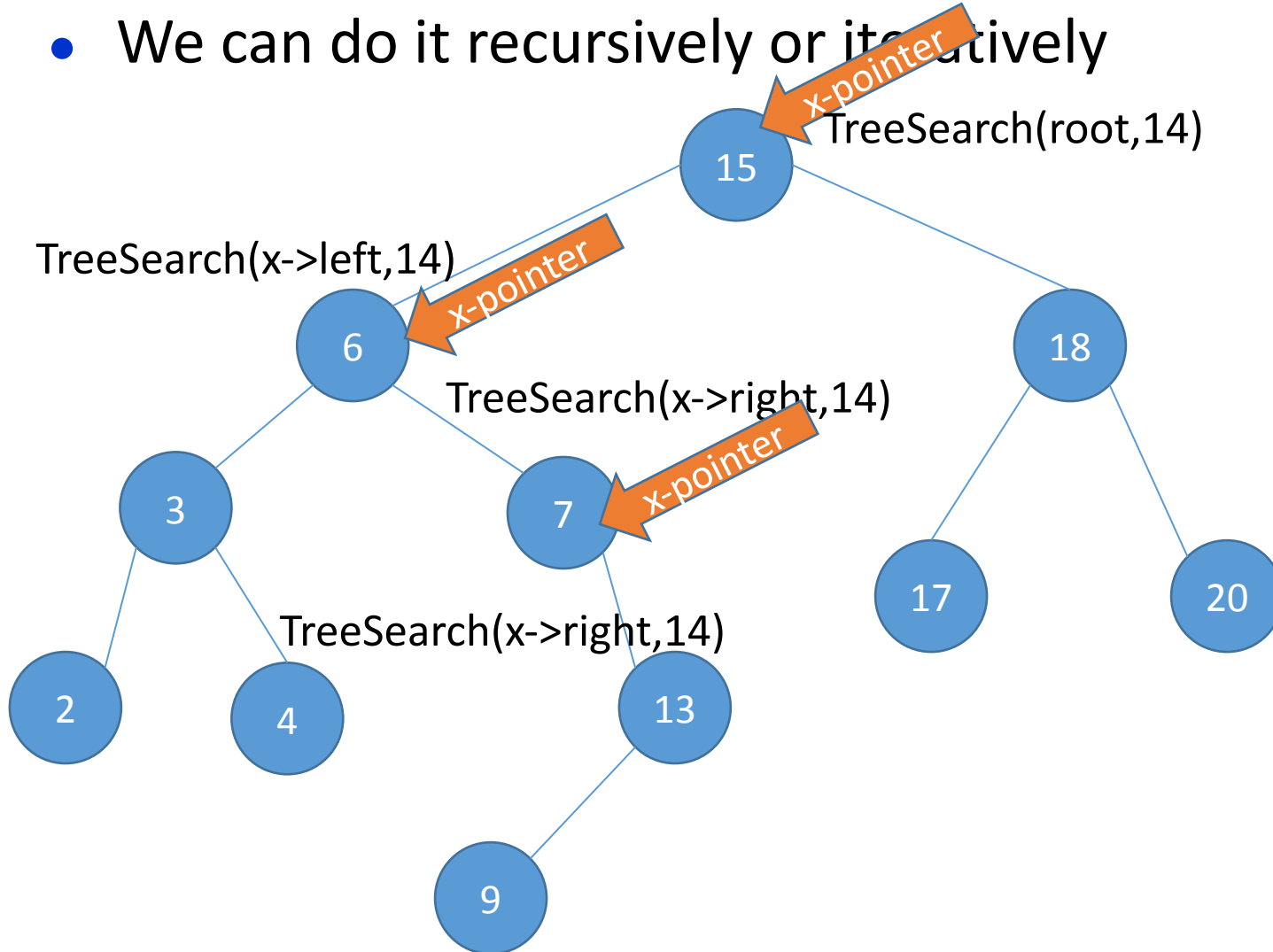
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

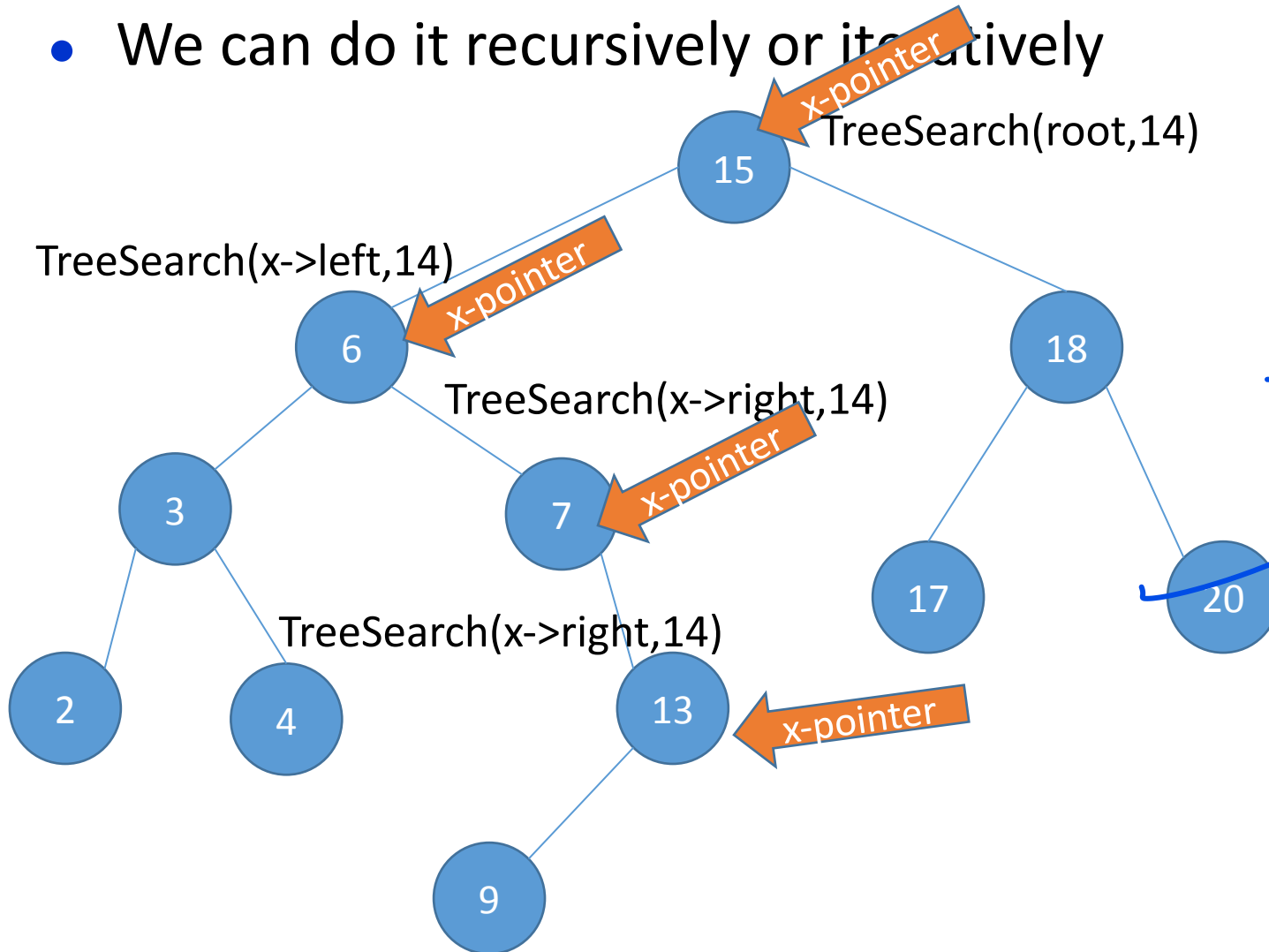
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

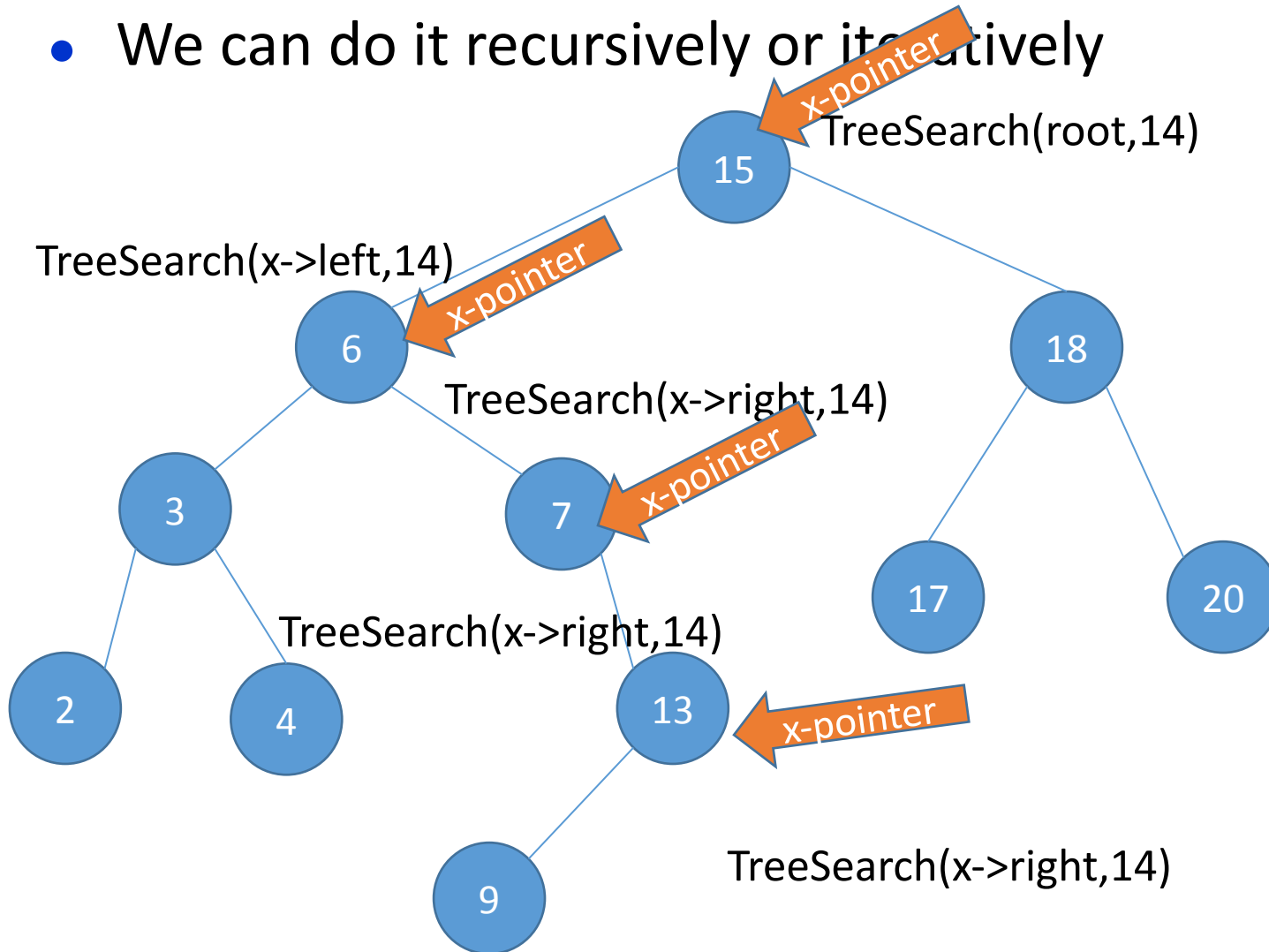
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

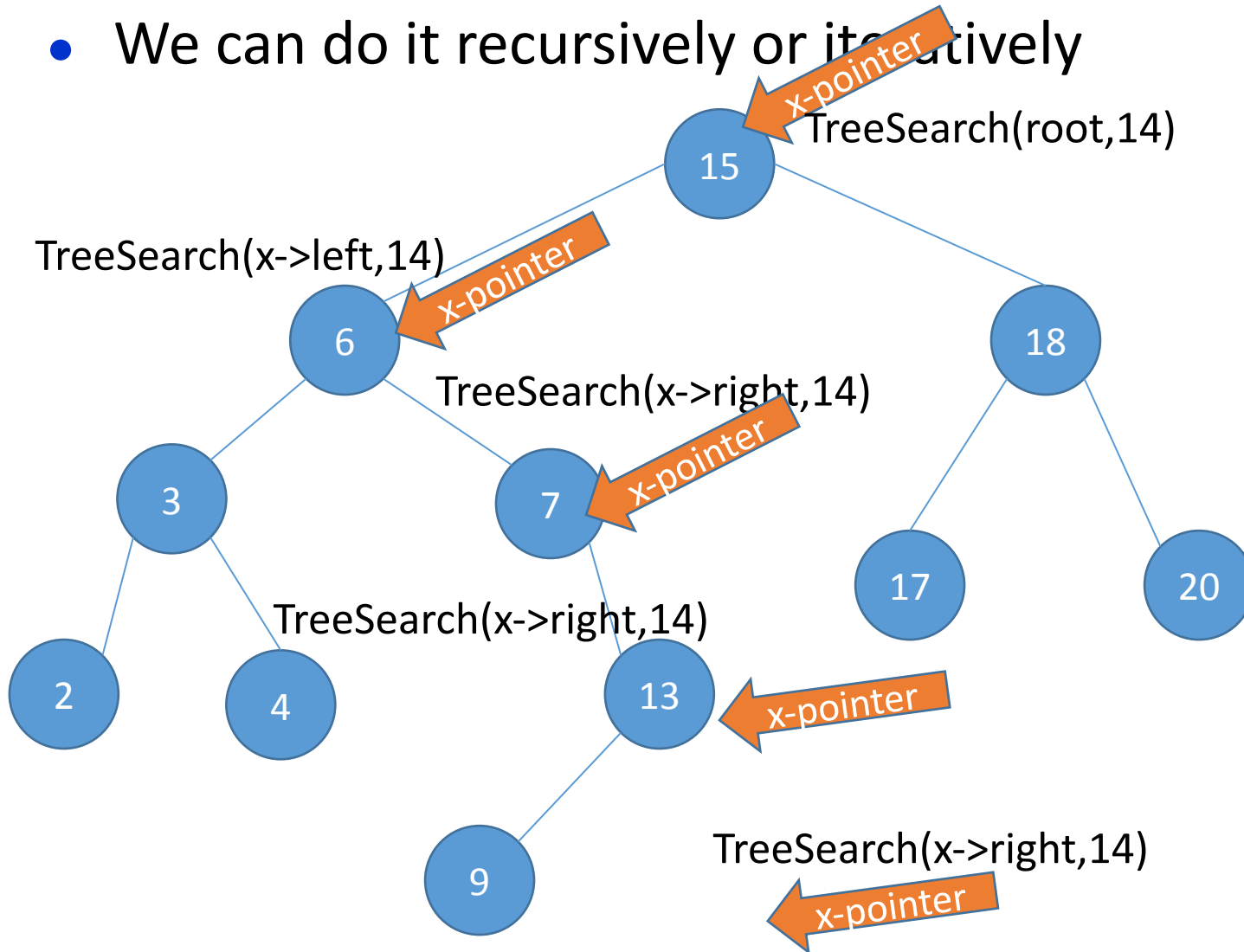
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

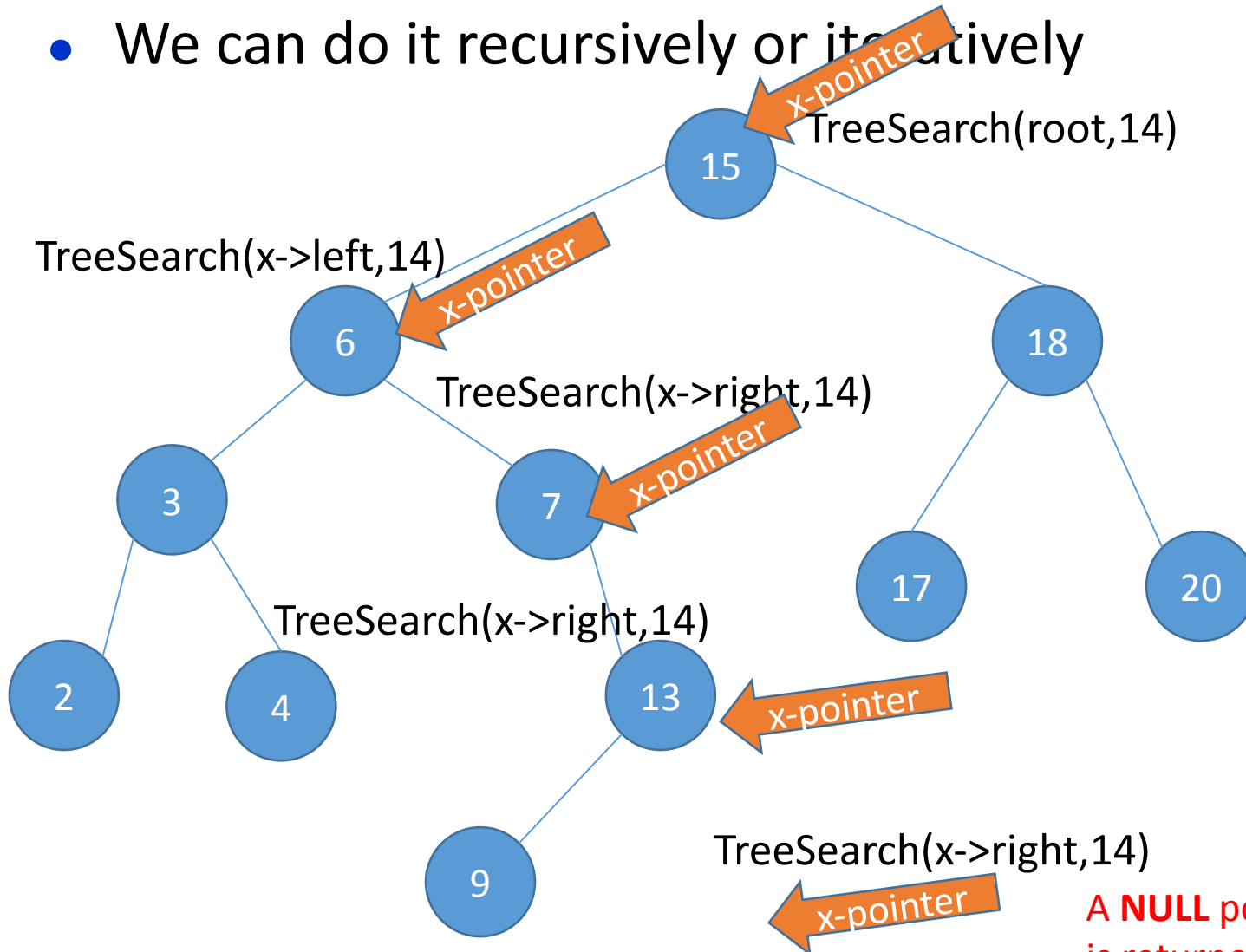
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

Search (Another Example)

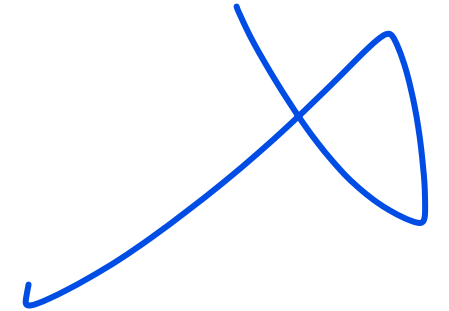
- We can do it recursively or iteratively



1. The function **takes a pointer** to a node and the value to be searched as input.
2. The function then places **a pointer x** at the same node.
3. Using that pointer the function checks if the **value is equal**.
 - If it is equal then a pointer to that node is returned.
4. If the **key is greater** than that value
 - The function calls another function on the **right child**
5. If the **key is smaller** than that value
 - The function calls another function on the **left child**

A **NULL** pointer
is returned

Search (Let us Write The Code)



Search (Let us Write The Code)

- Recursive

TREE-SEARCH(x, k)

```
1  if  $x == \text{NIL}$  or  $k == x.key$ 
2      return  $x$ 
3  if  $k < x.key$ 
4      return TREE-SEARCH( $x.left, k$ )
5  else return TREE-SEARCH( $x.right, k$ )
```

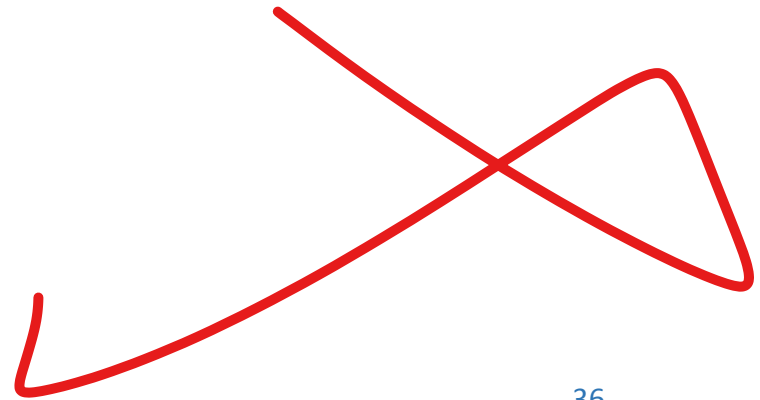


Search (Let us Write The Code)

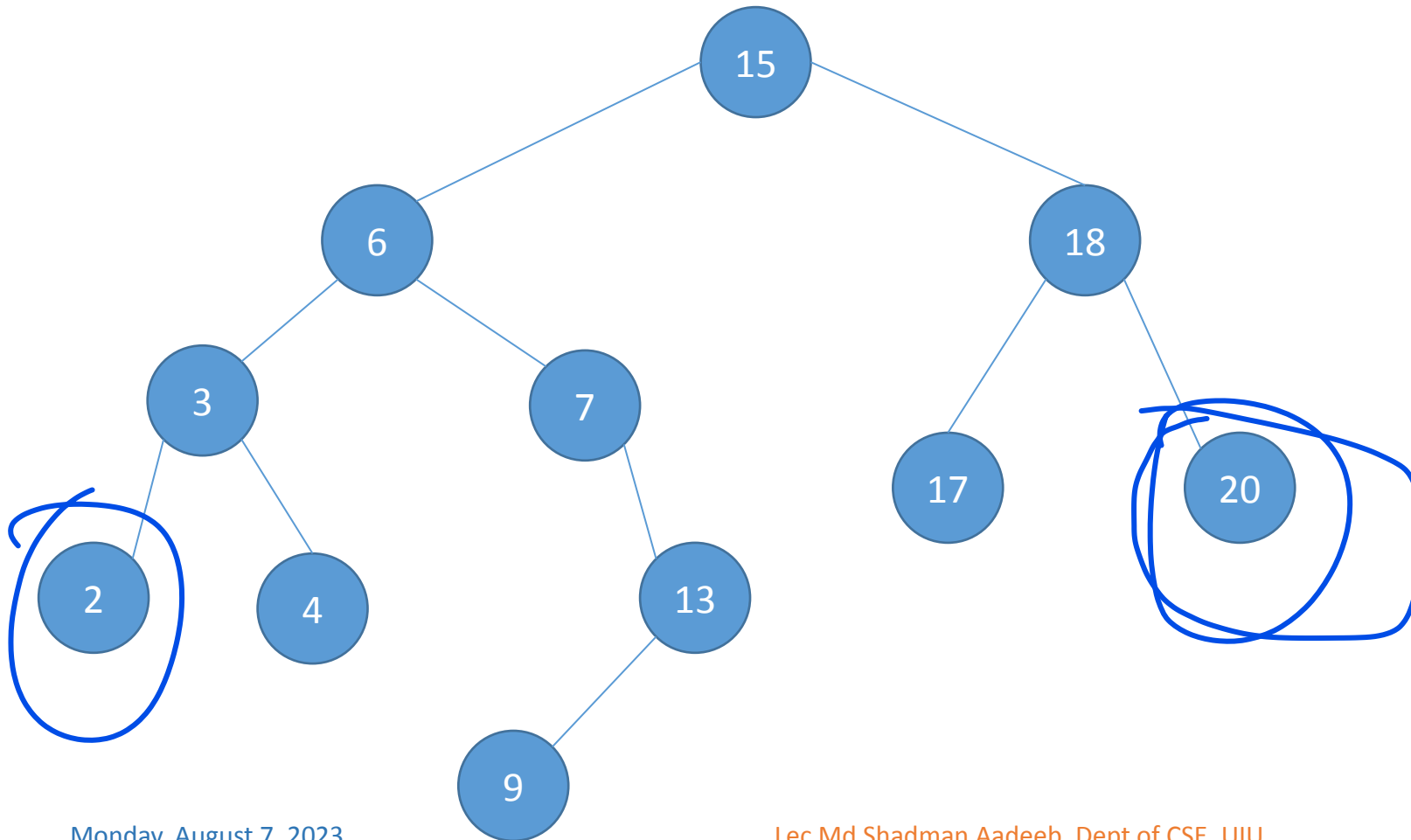
- Iterative
 - Simulate this code on the previous tree by yourself

ITERATIVE-TREE-SEARCH(x, k)

```
1  while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$ 
2      if  $k < x.\text{key}$ 
3           $x = x.\text{left}$ 
4      else  $x = x.\text{right}$ 
5  return  $x$ 
```

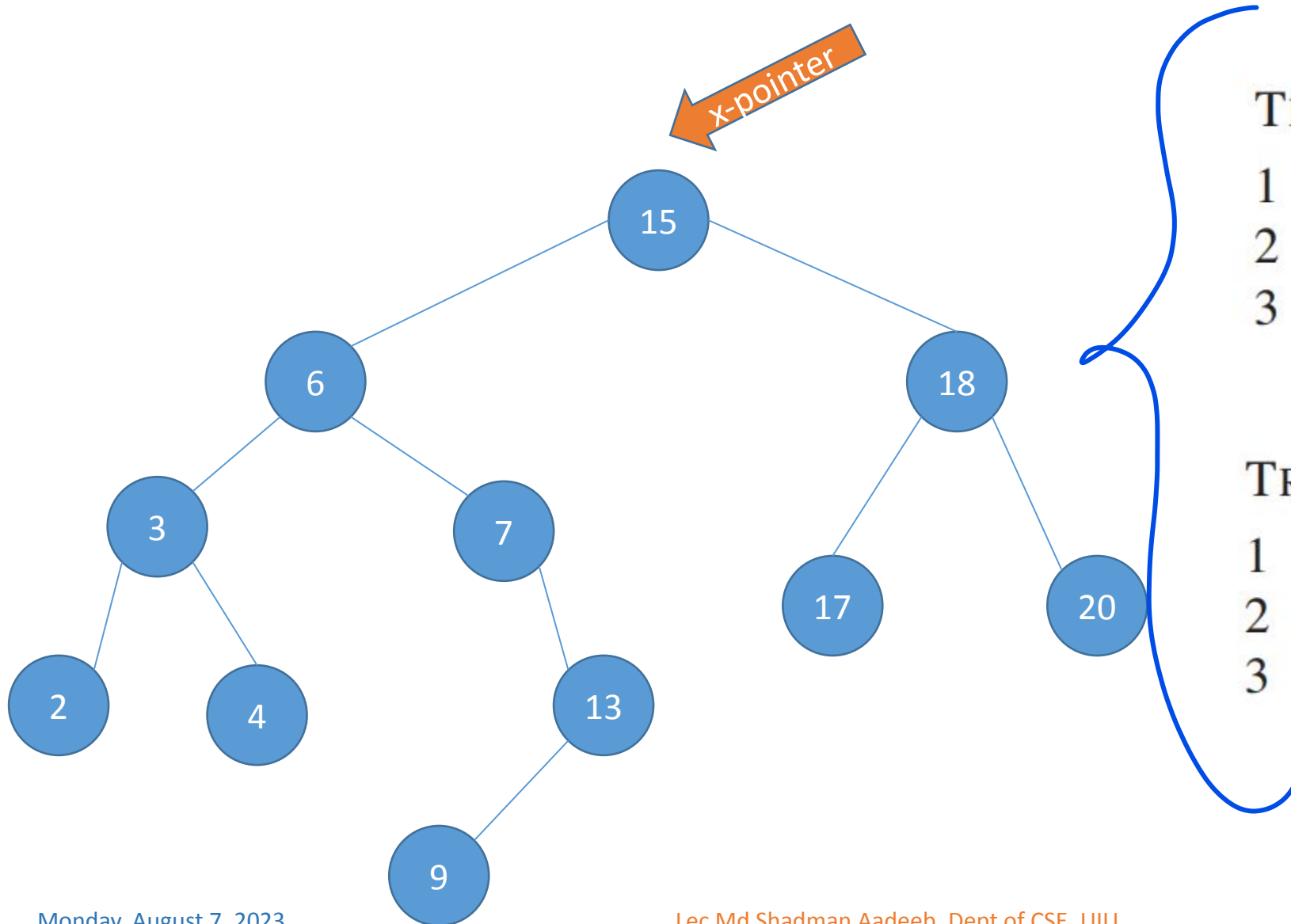


Finding the Minimum and Maximum Value



git

Finding the Minimum and Maximum Value



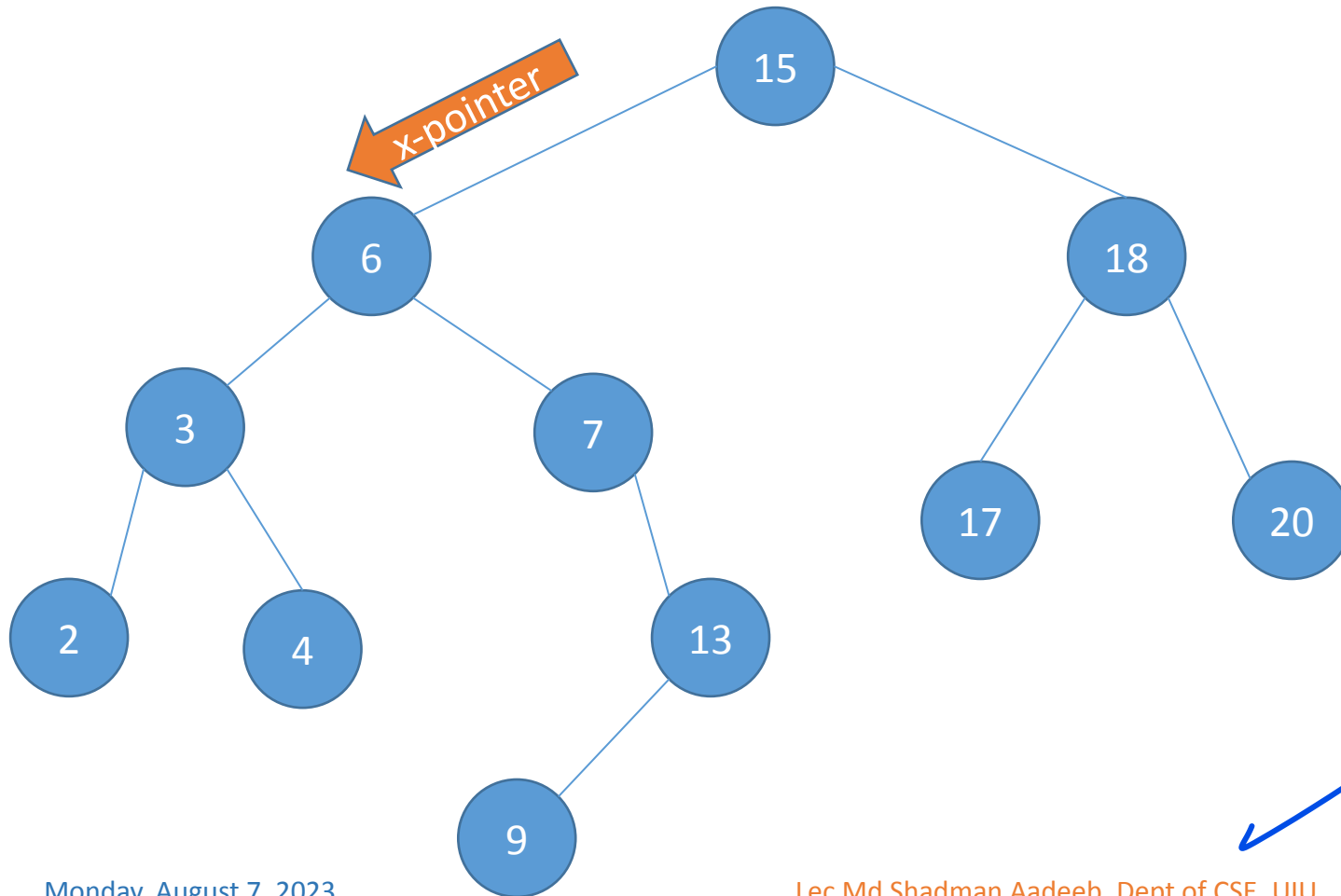
TREE-MINIMUM(x)

```
1  while  $x.left \neq \text{NIL}$ 
2       $x = x.left$ 
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $x.right \neq \text{NIL}$ 
2       $x = x.right$ 
3  return  $x$ 
```

Finding the Minimum and Maximum Value



TREE-MINIMUM(x)

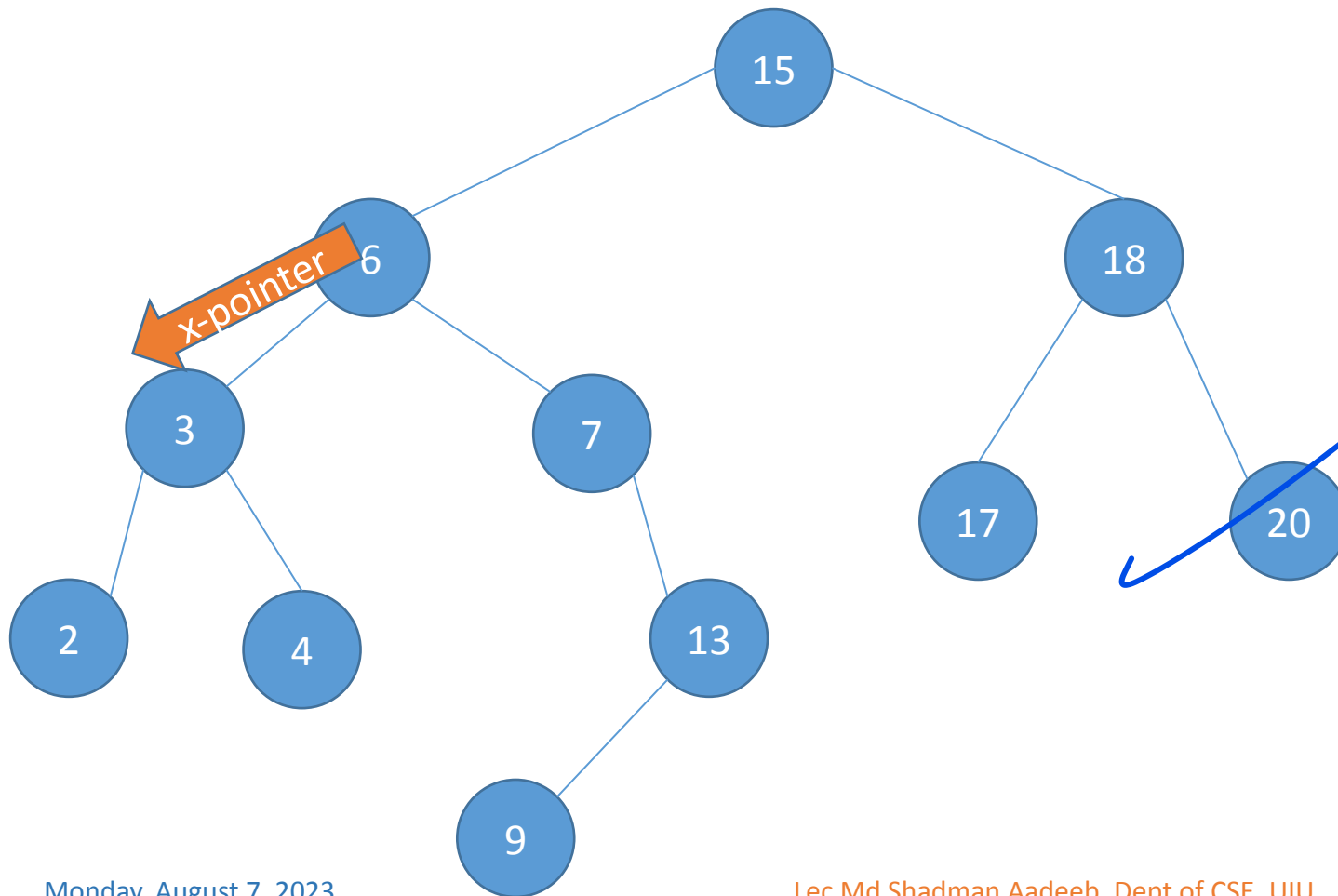
```
1  while  $x.left \neq \text{NIL}$ 
2       $x = x.left$ 
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $x.right \neq \text{NIL}$ 
2       $x = x.right$ 
3  return  $x$ 
```

A large blue 'X' is drawn over the TREE-MAXIMUM code block, indicating it is not the focus of the current slide.

Finding the Minimum and Maximum Value



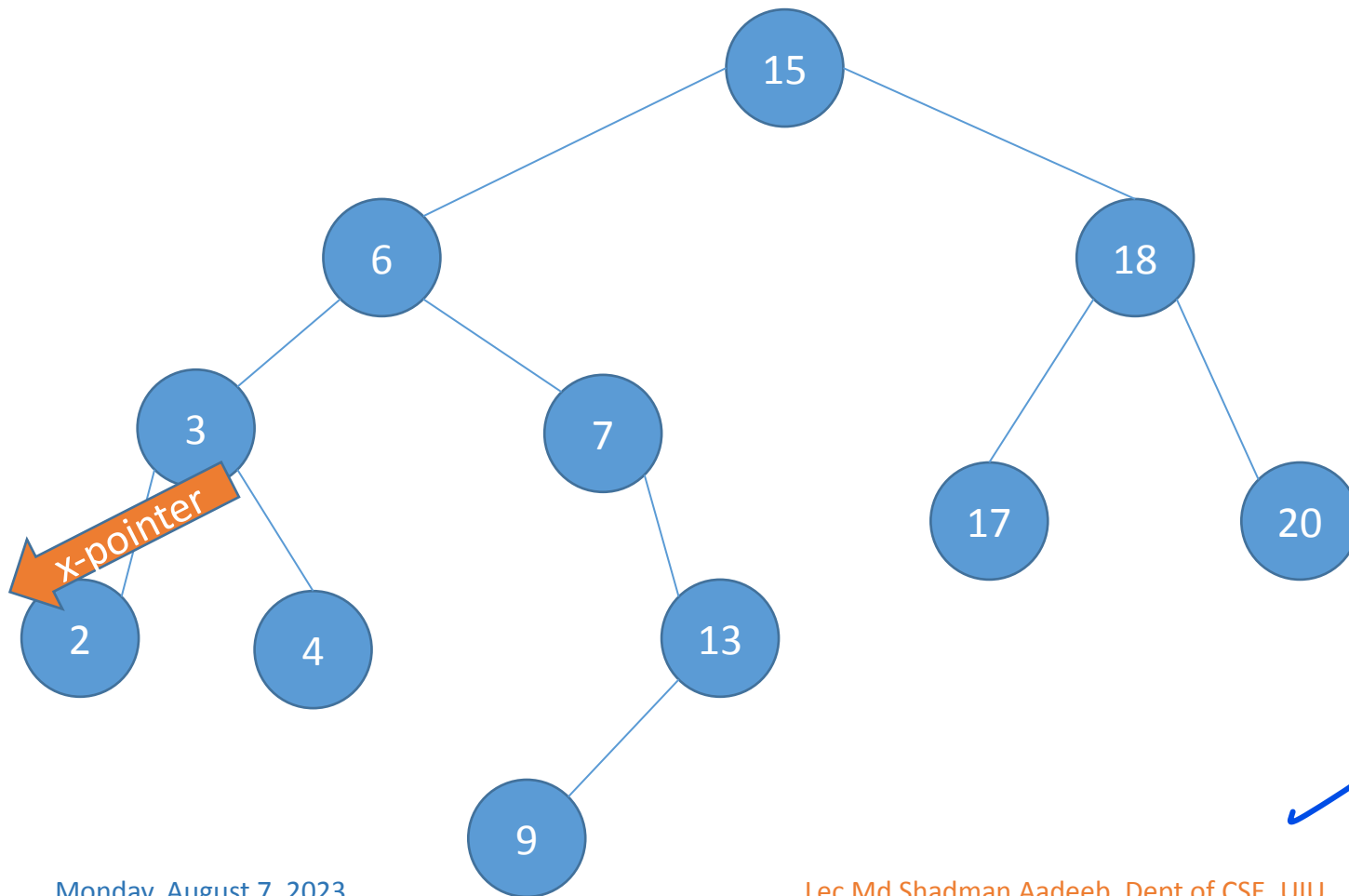
TREE-MINIMUM(x)

```
1  while  $x.left \neq \text{NIL}$ 
2       $x = x.left$ 
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $x.right \neq \text{NIL}$ 
2       $x = x.right$ 
3  return  $x$ 
```


Finding the Minimum and Maximum Value



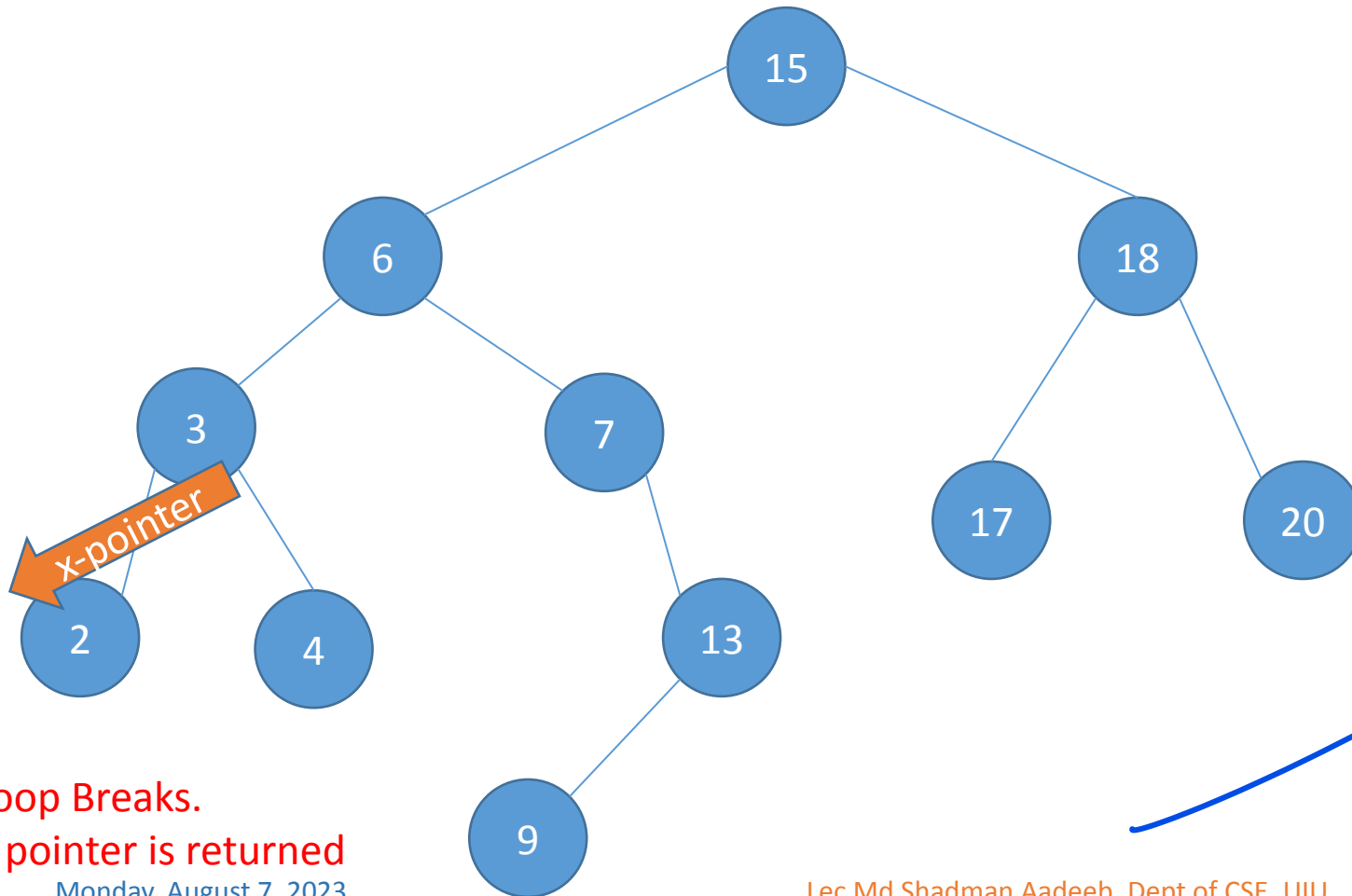
TREE-MINIMUM(x)

```
1  while  $x.left \neq \text{NIL}$ 
2       $x = x.left$ 
3  return  $x$ 
```

TREE-MAXIMUM(x)

```
1  while  $x.right \neq \text{NIL}$ 
2       $x = x.right$ 
3  return  $x$ 
```

Finding the Minimum and Maximum Value



TREE-MINIMUM(x)

```
1  while  $x.left \neq \text{NIL}$ 
2       $x = x.left$ 
3  return  $x$ 
```

TREE-MAXIMUM(x)

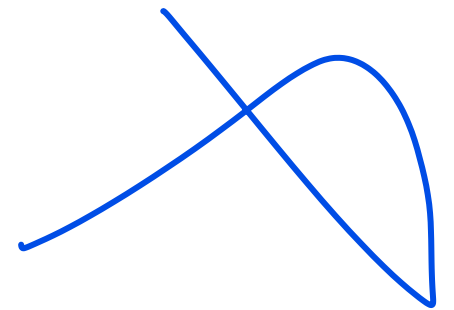
```
1  while  $x.right \neq \text{NIL}$ 
2       $x = x.right$ 
3  return  $x$ 
```

Loop Breaks.

X pointer is returned

Monday, August 7, 2023

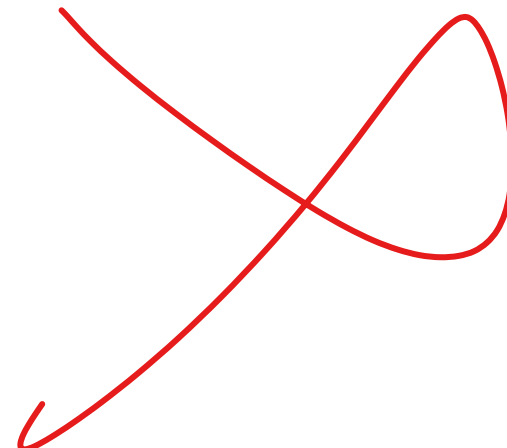
Successor and Predecessor



Successor and Predecessor

Successor of a node:

- If a node has a right child, the successor is the leftmost node in the right subtree.
- If a node does not have a right child, then the successor is the nearest ancestor of the node for which this node would be in its left subtree.



Successor and Predecessor

Value $\leftarrow$ (5)

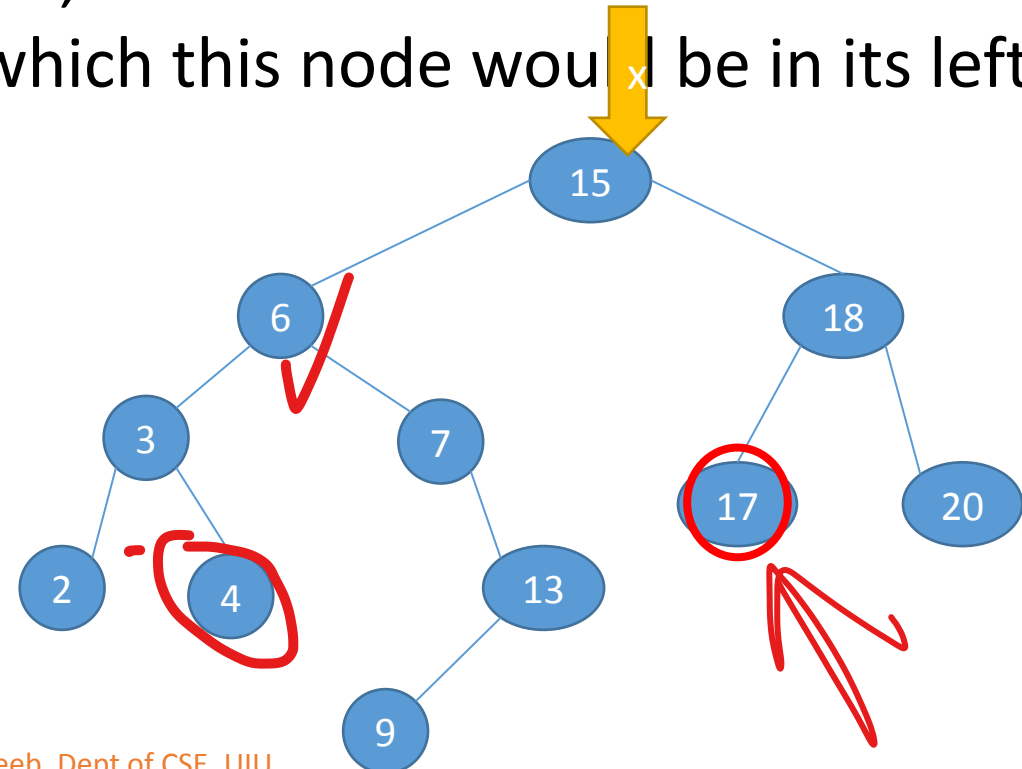
(p) $\leftarrow$ v

Successor of a node:

- If a node has a **right child**, the successor is the leftmost node in the right subtree.
- If a node does not have a right child, then the successor is the nearest ancestor of the node for which this node would be in its left subtree.

CASE 1: Node has a right child

Successor (root) - $\rightarrow$ 17



Successor and Predecessor

Successor of a node:

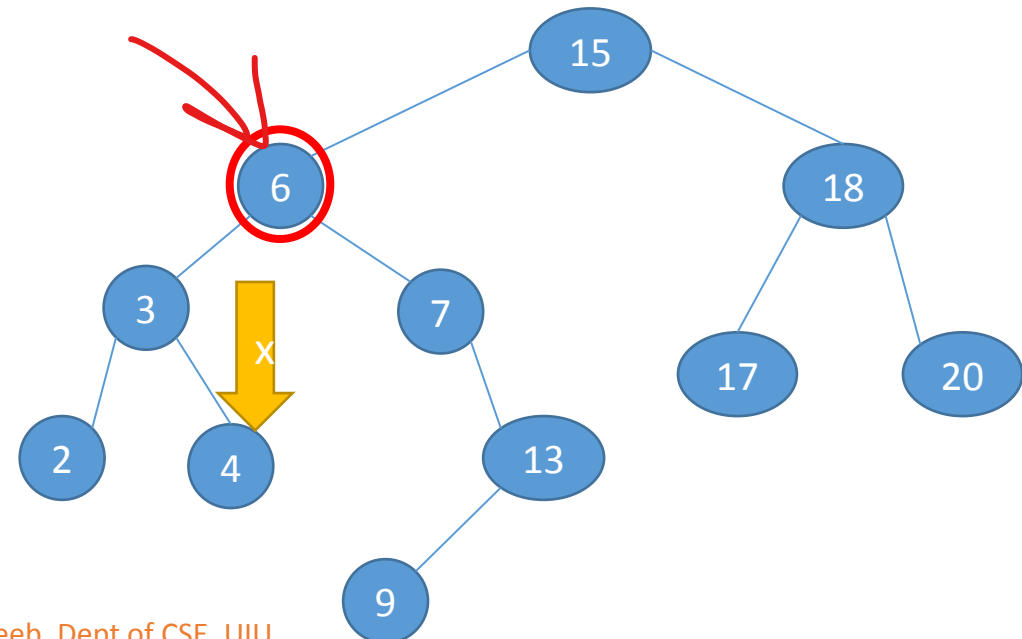
- If a node has a right child, the successor is the leftmost node in the right subtree.
- If a node **does not have a right child**, then the successor is the nearest **ancestor** of the node for which **this node** would be in its **left subtree**.

CASE 1: Node has a right child

Successor (root) - > 17

CASE 2: No Right Child

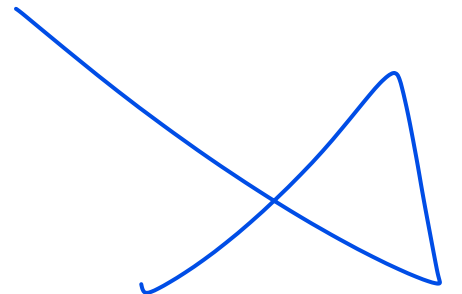
Successor (x) - > 6



Predecessor of a Node

Predecessor of a node (Mirror of the previous definition)

- If a node has a left child: The predecessor is the rightmost node in the left subtree.
- If a node does not have a left child: The predecessor is the nearest ancestor of the node for which this node would be in its right subtree.



Predecessor of a Node

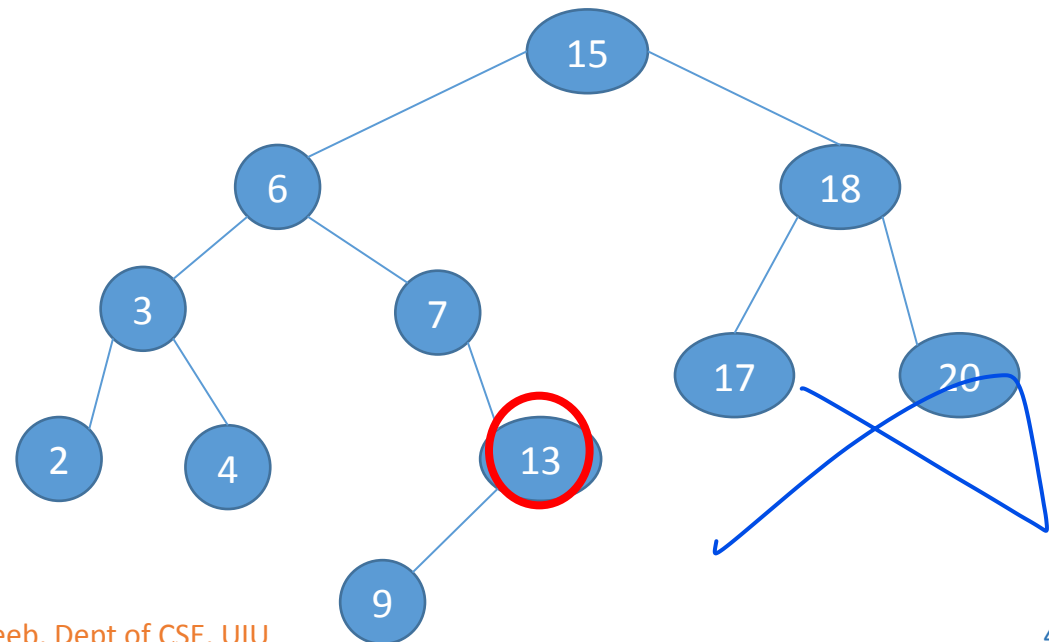
Predecessor of a node (Mirror of the previous definition)

- If a node has a **left child**: The predecessor is the **rightmost node** in the left subtree.
- If a node **does not have a left child**: The predecessor is the nearest ancestor of the node for which this node would **be in its right subtree**.

CASE 1: Node has a left child

Predecessor (root) - > 13

CASE 2: No Left Child





Predecessor of a Node

Predecessor of a node (Mirror of the previous definition)

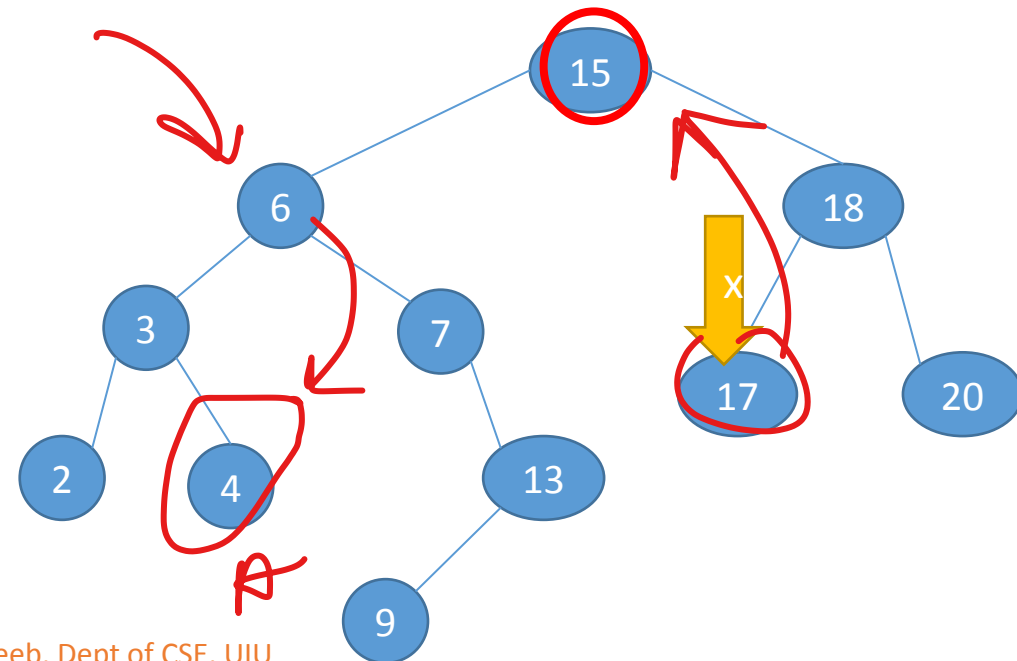
- If a node has a **left child**: The predecessor is the **rightmost node** in the left subtree.
- If a node **does not have a left child**: The predecessor is the nearest ancestor of the node for which this node would **be in its right subtree**.

CASE 1: Node has a left child

Predecessor (root) - > 13

CASE 2: No Left Child

Predecessor (x) - > 15

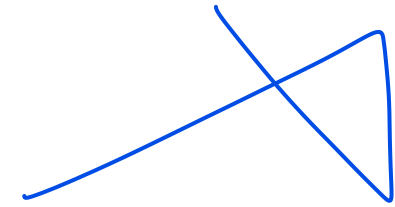
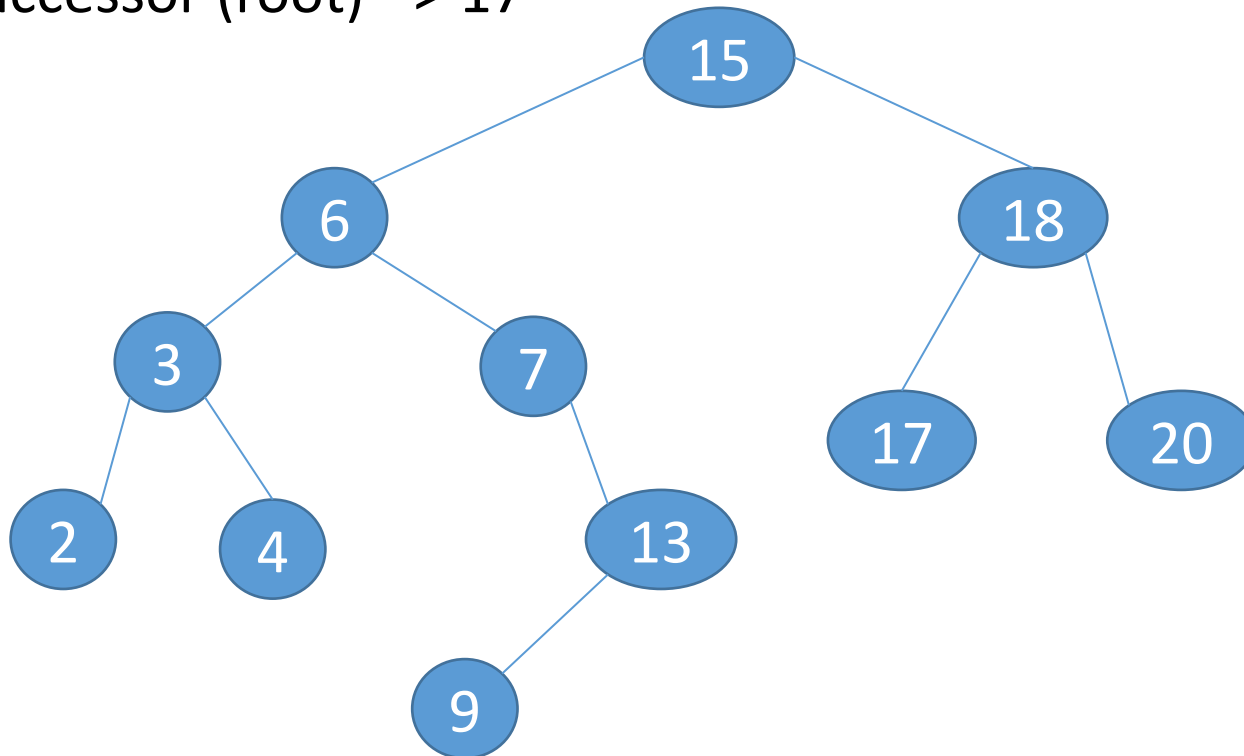


Determining the Successor : Simulation

CASE 1: Node has a right child

Leftmost node in the right subtree.

Successor (root) - > 17

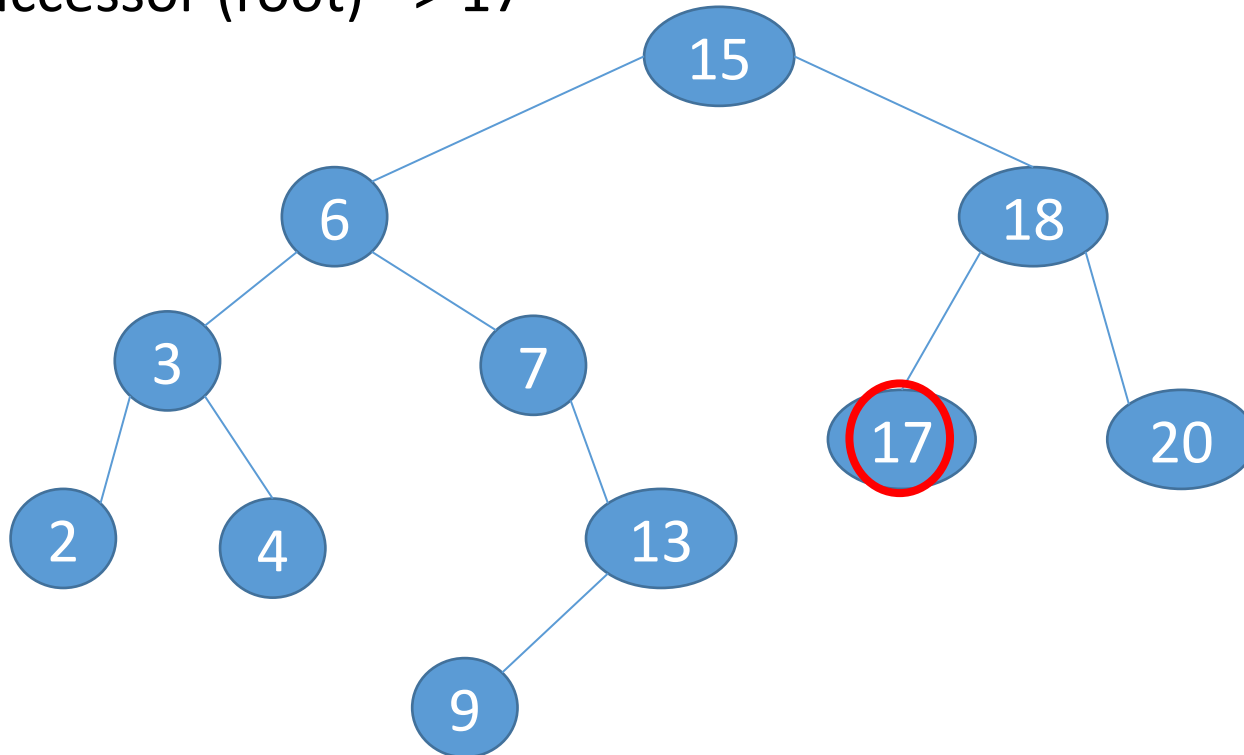


Determining the Successor : Simulation

CASE 1: Node has a right child

Leftmost node in the right subtree.

Successor (root) - > 17



TREE-SUCCESSOR(x)

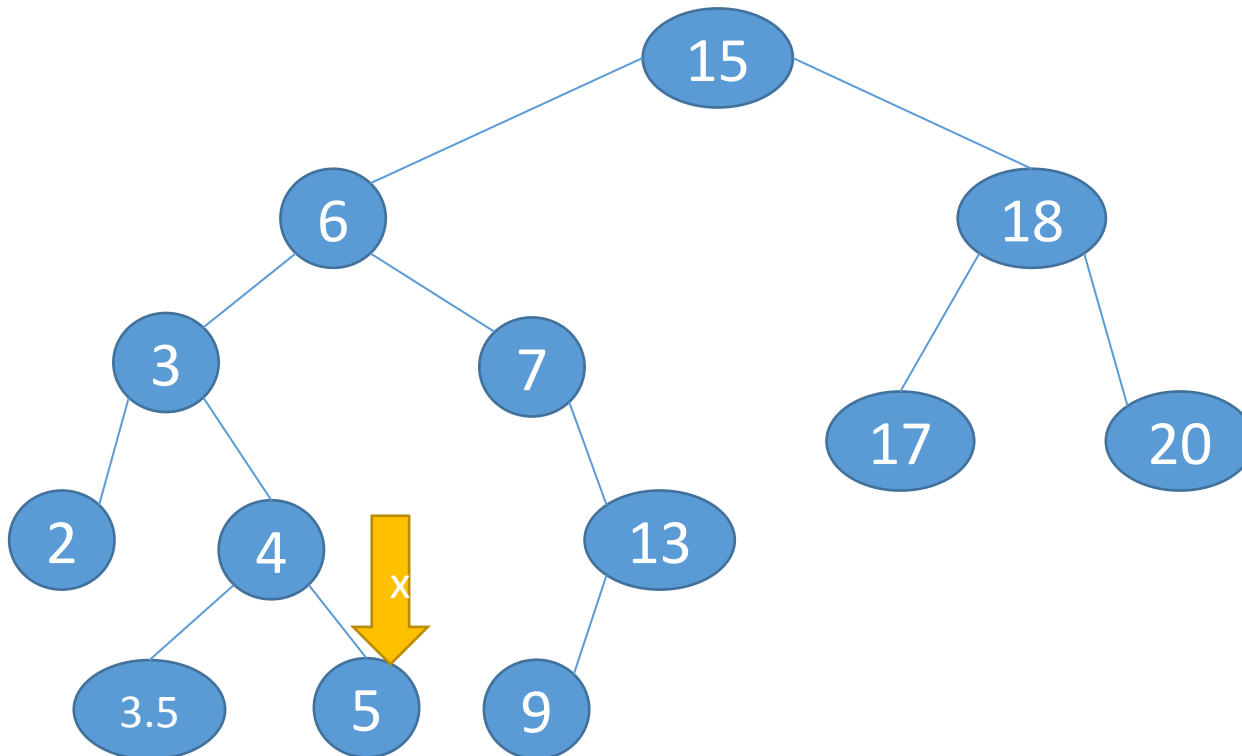
```
1  if  $x.right \neq \text{NIL}$   
2      return TREE-MINIMUM( $x.right$ )
```

Determining the Successor : Simulation

CASE 2: Node has no right child

The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6



TREE-SUCCESSOR(*x*)

```
1  if x.right ≠ NIL
2      return TREE-MINIMUM(x.right)
3  y = x.p
4  while y ≠ NIL and x == y.right
5      x = y
6      y = y.p
7  return y
```

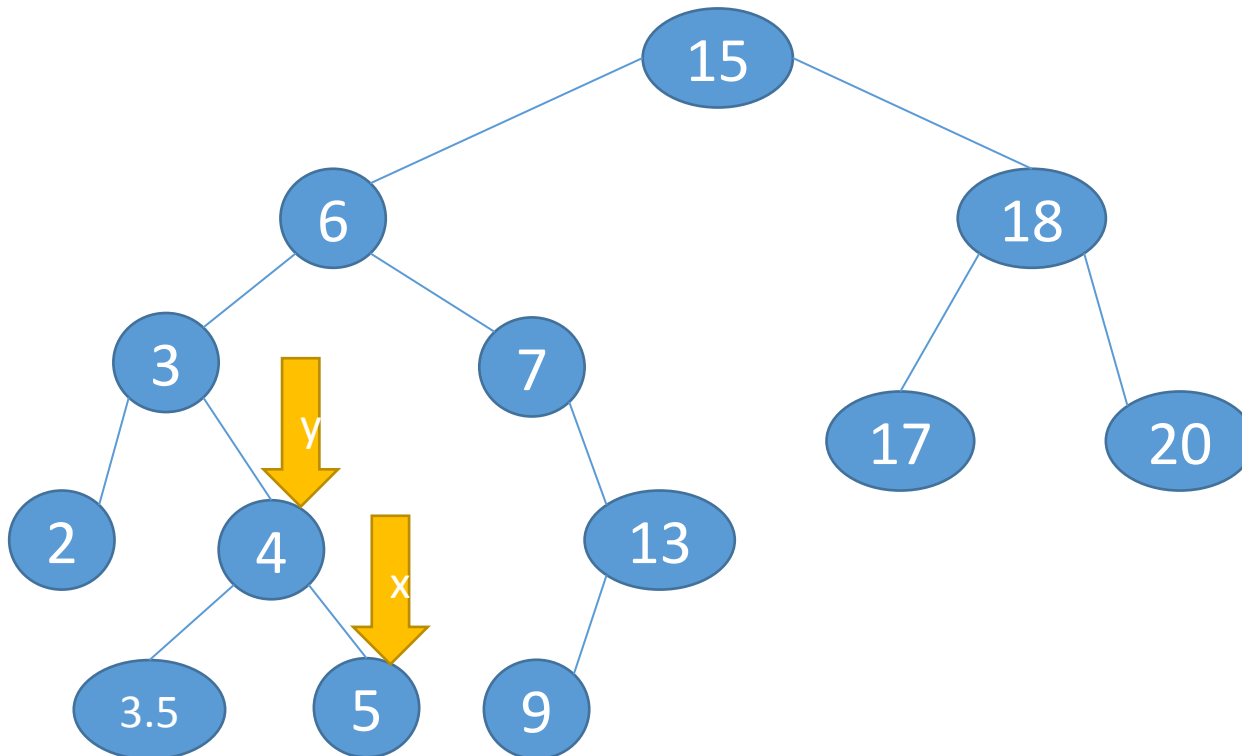
gift

Determining the Successor : Simulation

CASE 2: Node has no right child

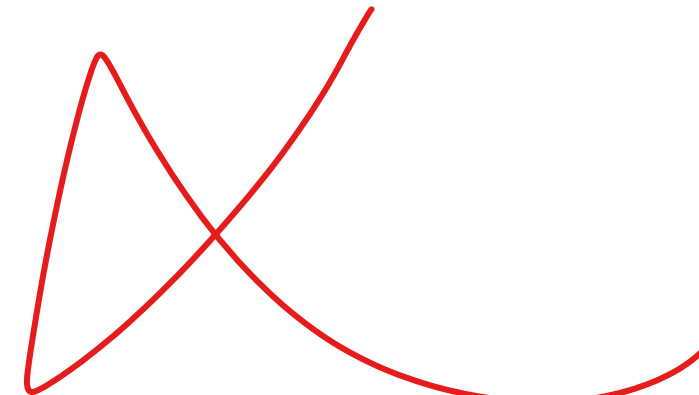
The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6



TREE-SUCCESSOR(x)

```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

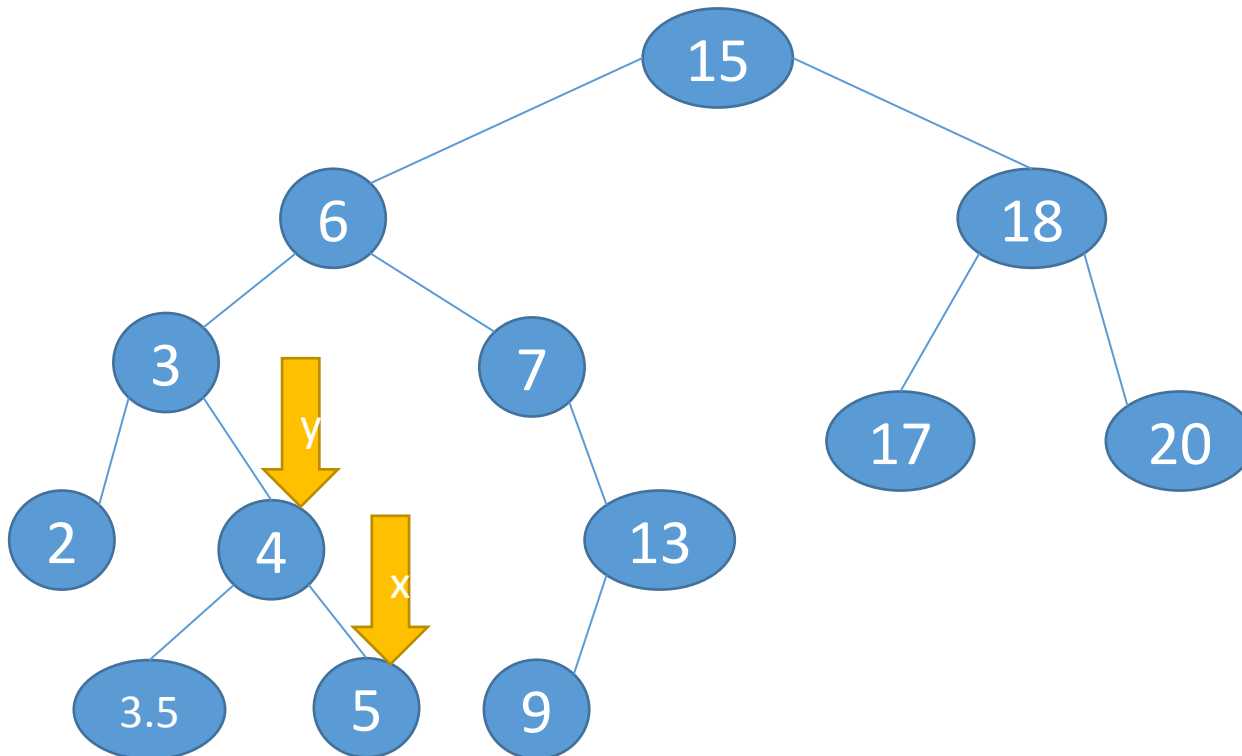


Determining the Successor : Simulation

CASE 2: Node has no right child

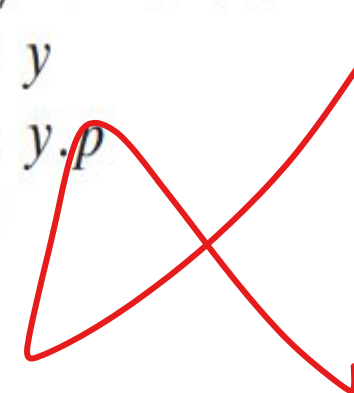
The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6



TREE-SUCCESSOR(x)

```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

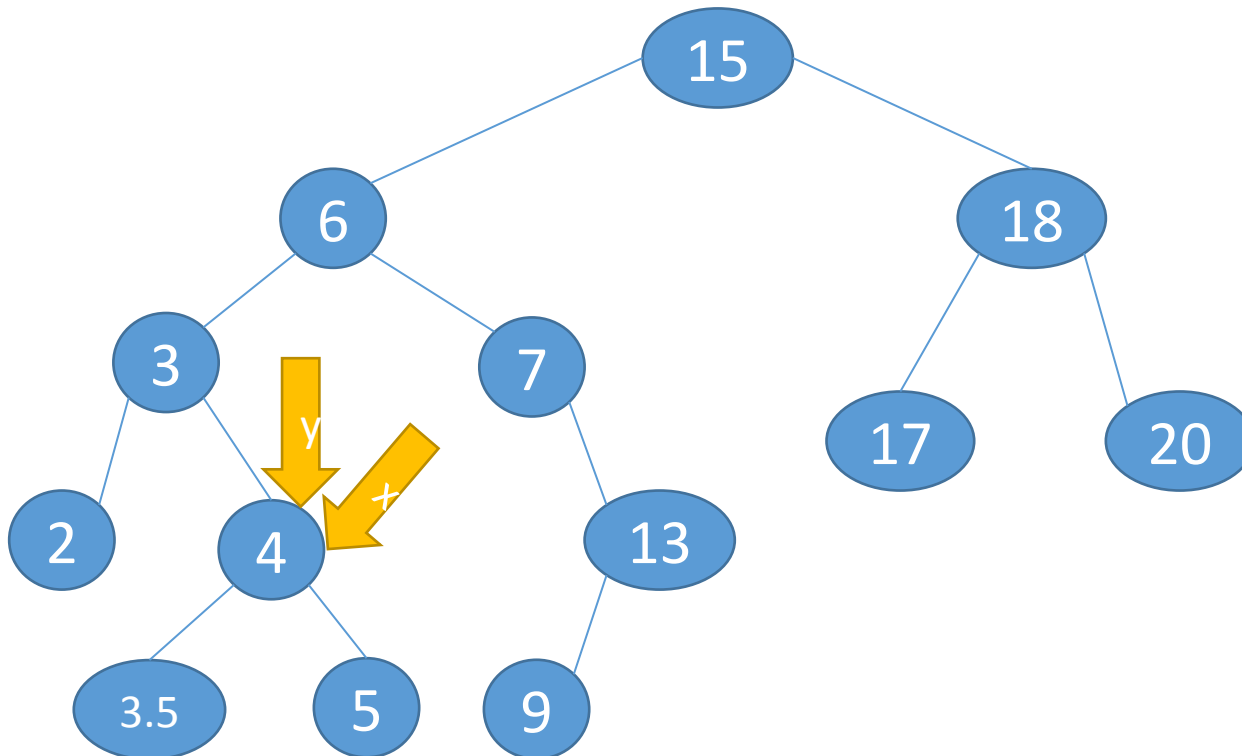


Determining the Successor : Simulation

CASE 2: Node has no right child

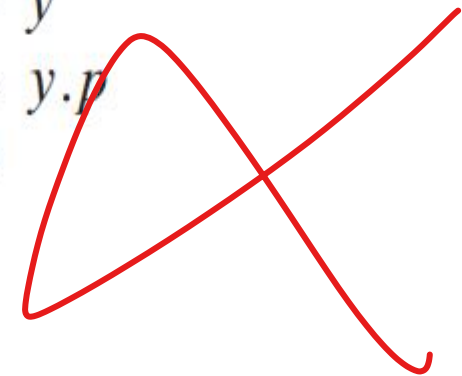
The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6



TREE-SUCCESSOR(*x*)

```
1  if x.right ≠ NIL
2      return TREE-MINIMUM(x.right)
3  y = x.p
4  while y ≠ NIL and x == y.right
5      x = y
6      y = y.p
7  return y
```

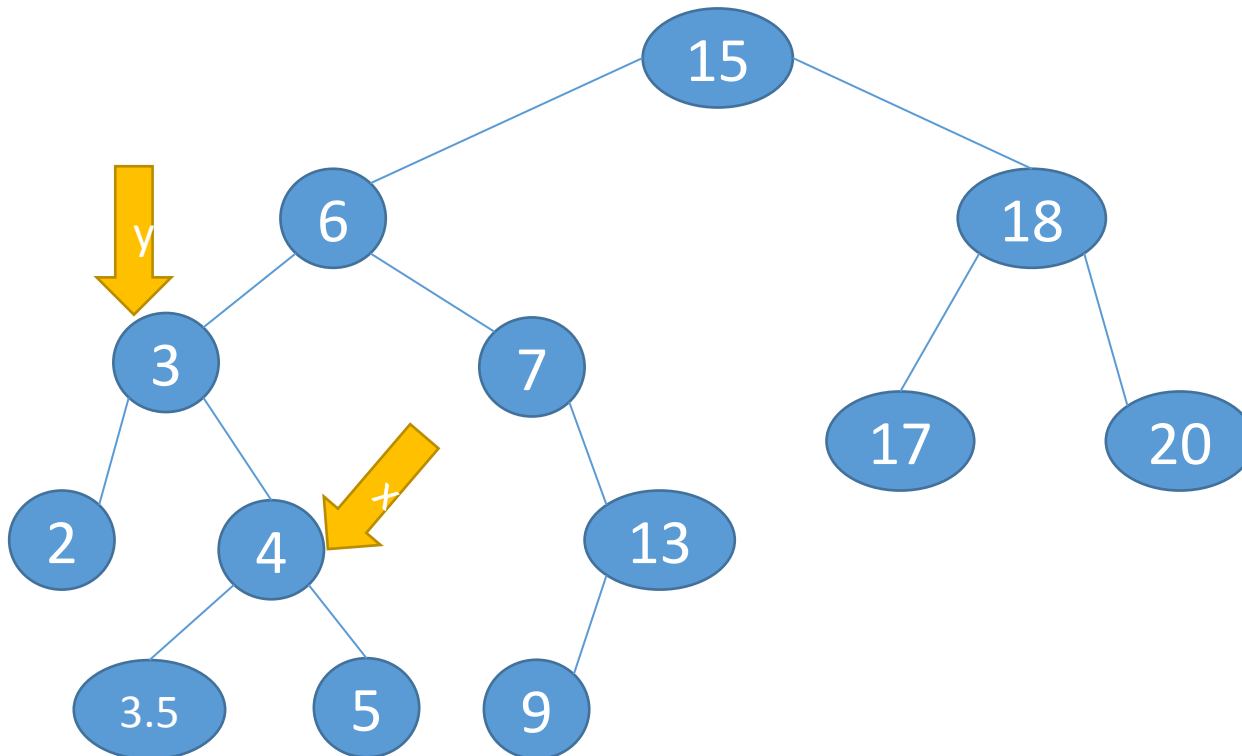


Determining the Successor : Simulation

CASE 2: Node has no right child

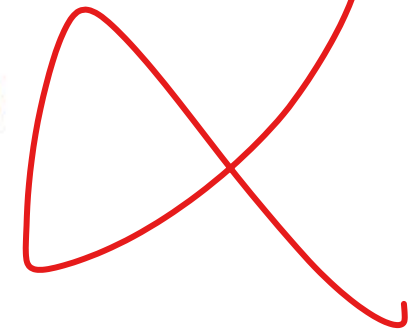
The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) -> 6



TREE-SUCCESSOR(x)

```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

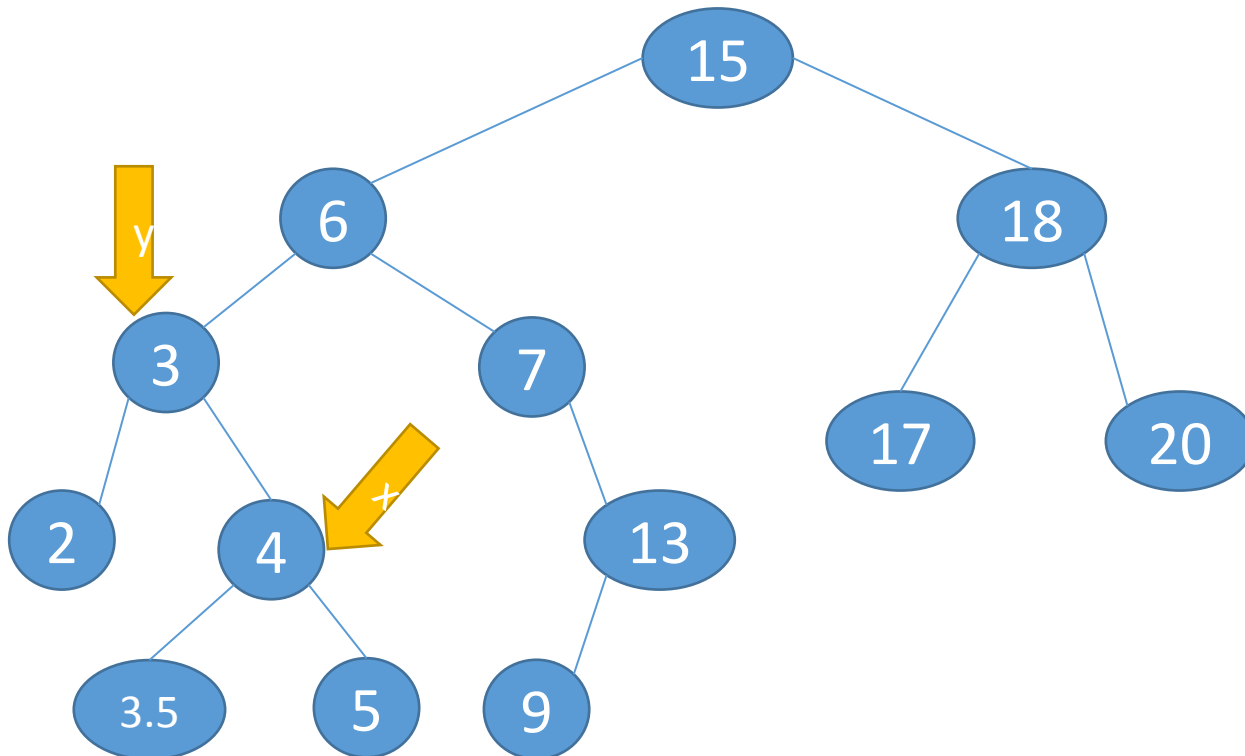


Determining the Successor : Simulation

CASE 2: Node has no right child

The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6



TREE-SUCCESSOR(x)

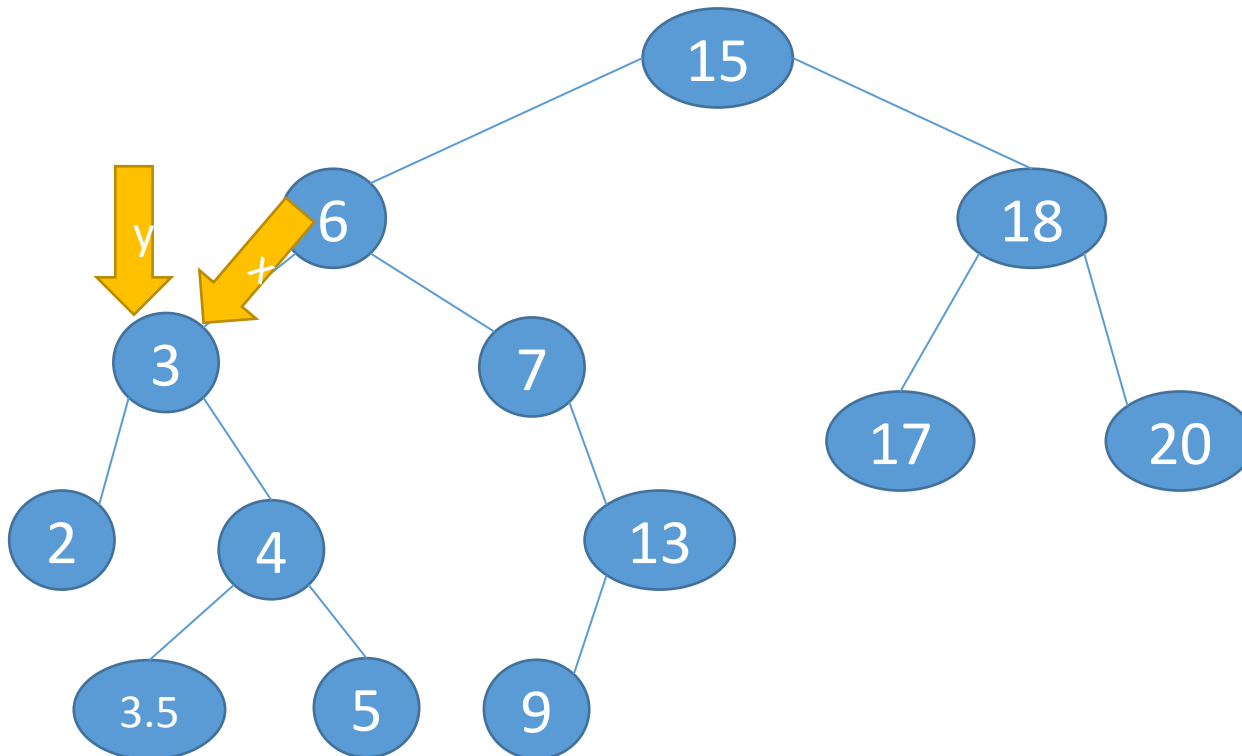
```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x \neq y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

Determining the Successor : Simulation

CASE 2: Node has no right child

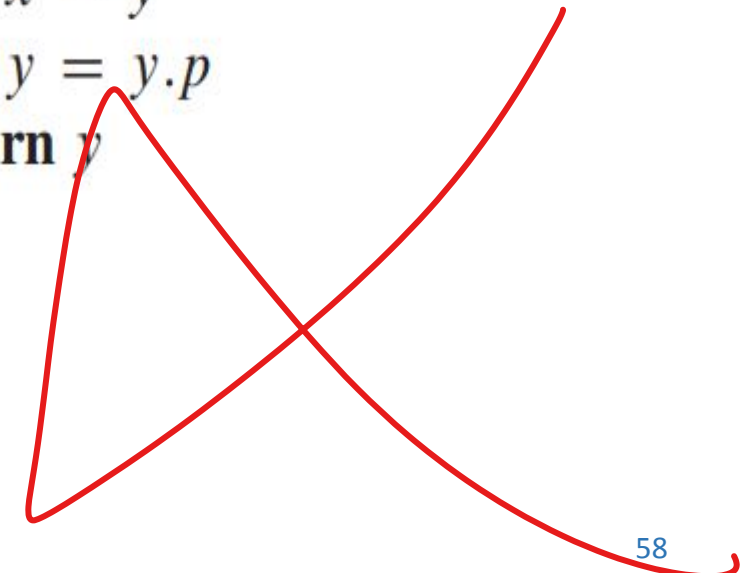
The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6



TREE-SUCCESSOR(x)

```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

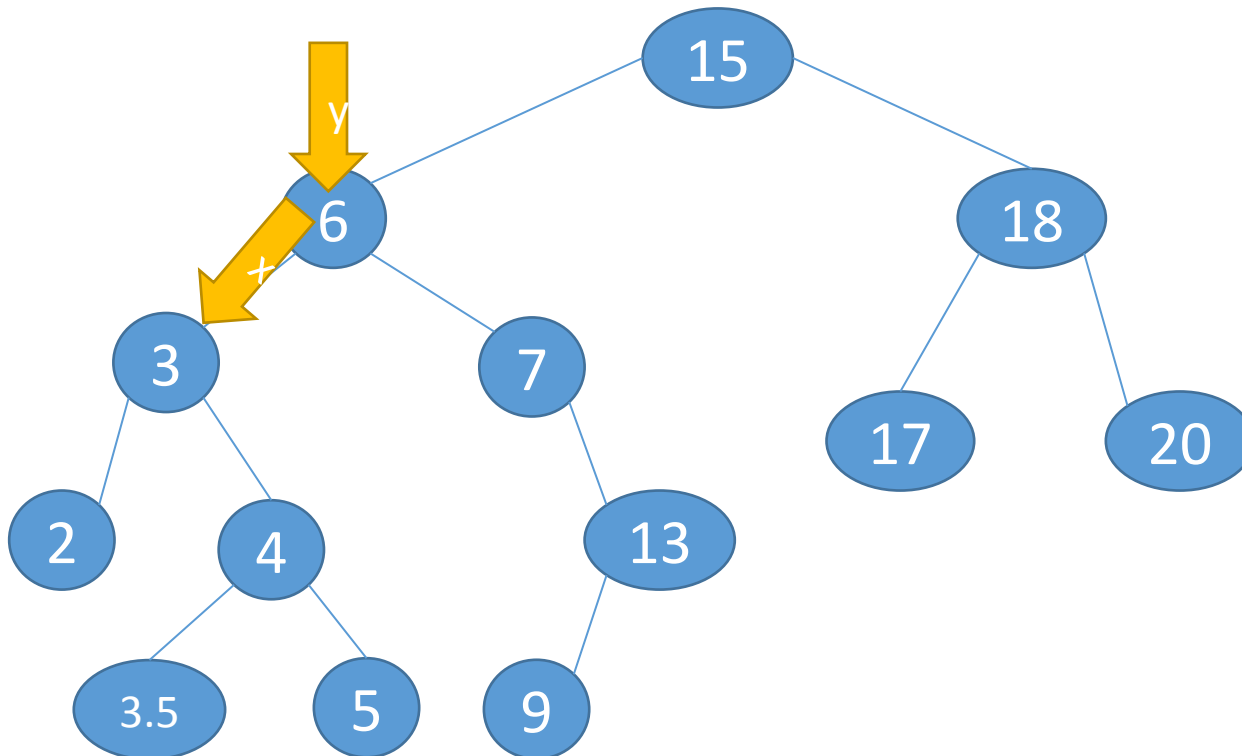


Determining the Successor : Simulation

CASE 2: Node has no right child

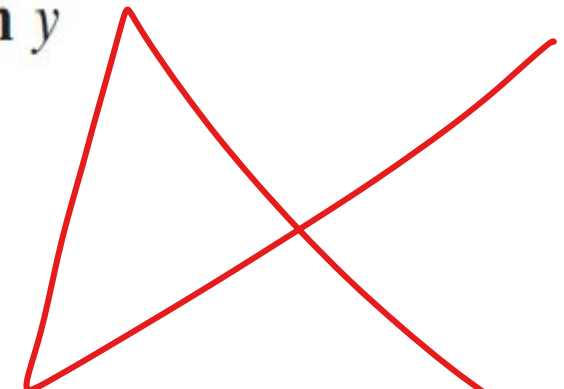
The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6



TREE-SUCCESSOR(x)

```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

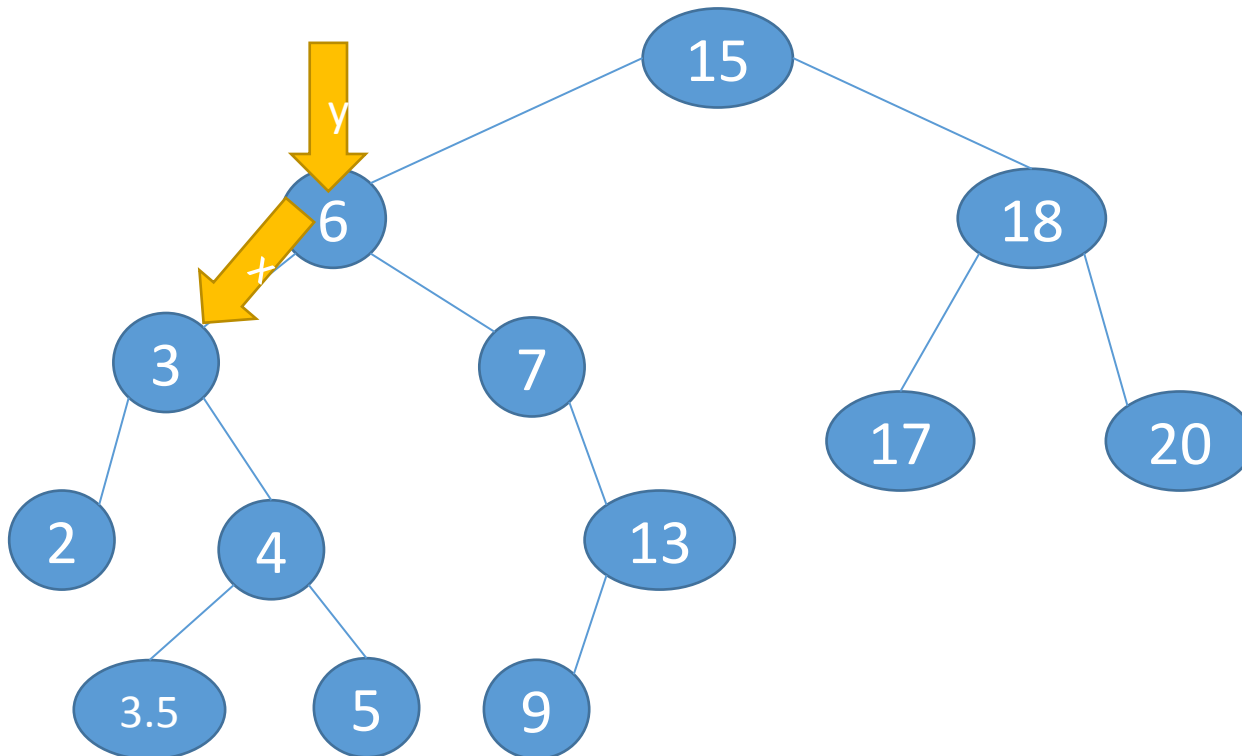


Determining the Successor : Simulation

CASE 2: Node has no right child

The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) -> 6



TREE-SUCCESSOR(x)

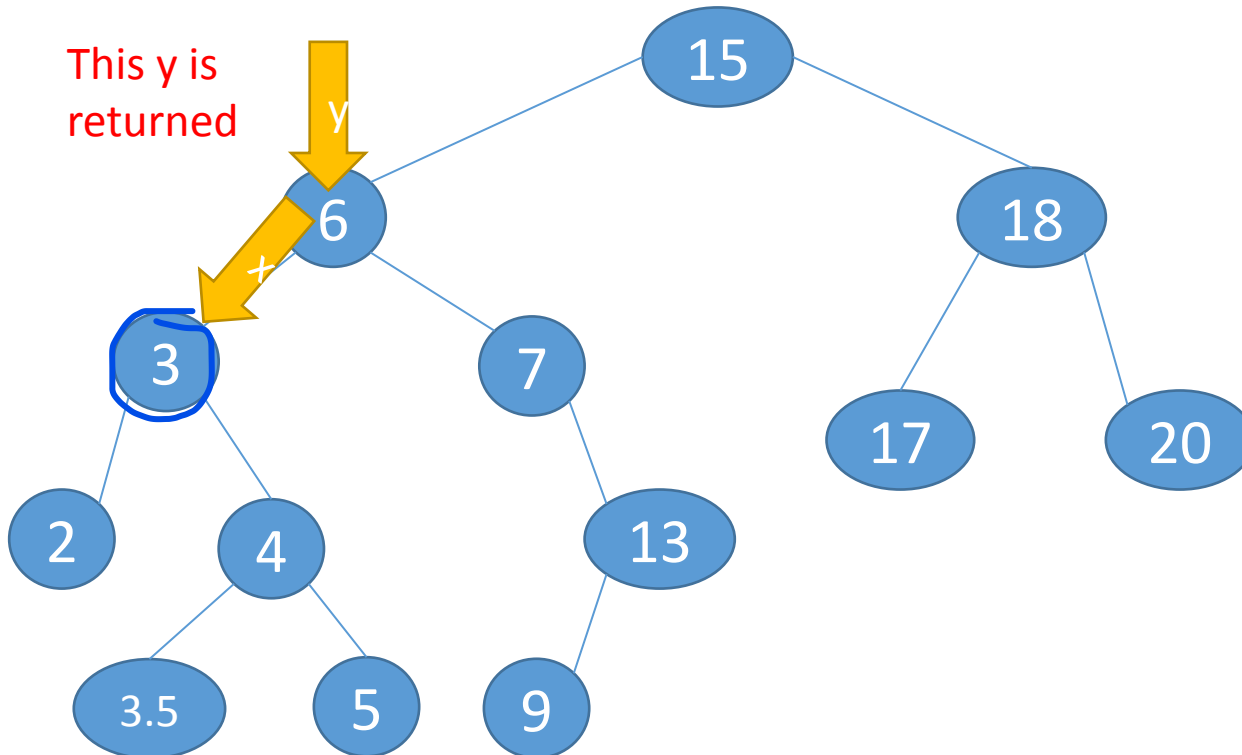
```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

Determining the Successor : Simulation

CASE 2: Node has no right child

The nearest ancestor of the node for which this node would be in its right subtree.

Successor (x) - > 6

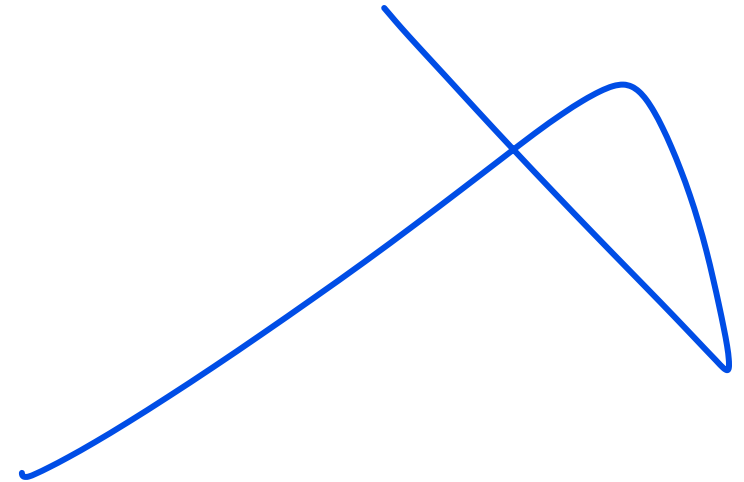


TREE-SUCCESSOR(x)

```
1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  and  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 
```

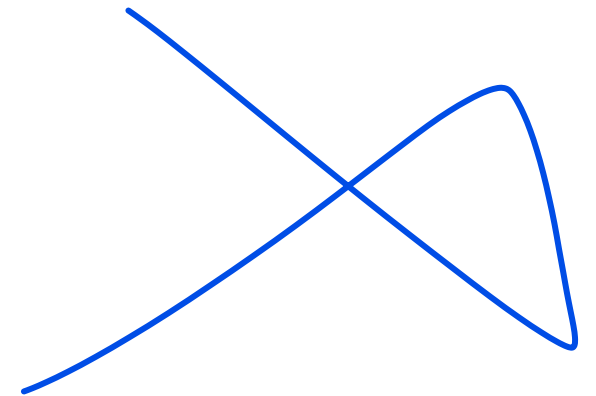
An orange arrow points from the tree diagram to the algorithm. A red checkmark is drawn over the algorithm, indicating it is correct.

Write down a similar code for predecessor



Next Class

- Discussing Complexity
- Insert Function
- Delete Function



Thank You

